

UNIVERSITAS GUNADARMA
FAKULTAS TEKNOLOGI INDUSTRI



SKRIPSI

IMPLEMENTASI KOMPONEN *ML SERVICE* DAN ANALISIS *LLM* PADA
PLATFORM *AUTOMATED MACHINE LEARNING* BERBASIS WEBSITE

Disusun Oleh:

Nama : Muhammad Rayhan Arliansyah
NPM : 51422133
Program Studi : Informatika
Pembimbing : Dr. rer. nat. I Made Wiryana, S.Si., S.Kom., M.Sc.

Diajukan Guna Melengkapi Sebagian Syarat Dalam Mencapai

Gelar Sarjana Strata Satu (S1)

JAKARTA

2025

PERNYATAAN ORIGINALITAS DAN PUBLIKASI

Saya yang bertanda tangan di bawah ini:

Nama : Muhammad Rayhan Arliansyah
NPM : 51422133
Judul Skripsi : IMPLEMENTASI KOMPONEN *ML SERVICE* DAN
ANALISIS *LLM* PADA PLATFORM *AUTOMATED
MACHINE LEARNING* BERBASIS WEBSITE
Tanggal Sidang : 16 September 2025
Tanggal Lulus : 16 September 2025

Menyatakan bahwa tulisan ini adalah merupakan hasil karya saya sendiri dan dapat dipublikasikan sepenuhnya oleh Universitas Gunadarma. Segala kutipan dalam bentuk apapun telah mengikuti kaidah dan etika yang berlaku. Semua hak cipta dari logo serta produk yang disebut dalam buku ini adalah milik masing-masing pemegang haknya, kecuali disebutkan lain. Mengenai isi dan tulisan merupakan tanggung jawab Penulis, bukan Universitas Gunadarma.

Demikianlah pernyataan ini dibuat dengan sebenarnya dan dengan penuh kesadaran.

Jakarta, 16 September 2025

(Muhammad Rayhan Arliansyah)

LEMBAR PENGESAHAN

Judul Skripsi : IMPLEMENTASI KOMPONEN *ML SERVICE* DAN
ANALISIS *LLM* PADA PLATFORM *AUTOMATED
MACHINE LEARNING* BERBASIS WEBSITE

Nama : Muhammad Rayhan Arliansyah

NPM : 51422133

Tanggal Lulus : 16 September 2025

PANITIA UJIAN

No.	Nama	Kedudukan
1.	Dr. Ravi Ahmad Salim, SSi.	Ketua
2.	Prof. Dr., Wahyudi Priyono Suwarso.	Sekretaris
3.	Dr. I Komang Sugiarta, SKom., MMSI.	Anggota
4.	Dr. Ida Astuti, SKom., MMSI.	Anggota
5.	Dr. Miftahul Jannah, SKom., MMSI.	Anggota

Menyetujui,

Pembimbing Bagian Sidang Ujian

(Dr. I Komang Sugiarta, S.Kom, MMSI.)

(Dr. Edi Sukirman, SSi., MM., M.I.Kom.)

ABSTRAK

MUHAMMAD RAYHAN ARLIANSYAH. 51422133.

IMPLEMENTASI KOMPONEN ML SERVICE DAN ANALISIS LLM PADA PLATFORM AUTOMATED MACHINE LEARNING BERBASIS WEBSITE. Skripsi. Informatika, Fakultas Ilmu Komputer dan Teknologi Informasi, Universitas Gunadarma, 2026.

Kata Kunci: ML Service, Automated Machine Learning, preprocessing, Scikit-learn, Large Language Model, Ollama, ROUGE.

Penelitian ini mengimplementasikan komponen ML Service dan pipeline analisis LLM pada platform Automated Machine Learning berbasis website. Komponen ML Service mendukung tiga jalur preprocessing: data tabular menggunakan ColumnTransformer, imputasi, penskalaan, one-hot encoding, dan LabelEncoder; data gambar menggunakan resize dan Histogram of Oriented Gradients; serta data teks menggunakan cleaning, stopword opsional, dan TF-IDF. Modul training menyediakan pemilihan algoritma, pembagian data, evaluasi metrik klasifikasi atau regresi, dan pengemasan model bersama artefak preprocessing. Sistem juga mengintegrasikan analisis teks berbasis model bahasa besar lokal melalui Ollama menggunakan few-shot prompting, parsing keluaran, pengukuran waktu generate, rata-rata panjang analisis, dan ROUGE. Pekerjaan panjang dijalankan melalui background task FastAPI agar API dapat mengembalikan identitas pekerjaan lebih awal. Hasil implementasi menunjukkan bahwa pipeline dapat dipanggil secara modular dan menghasilkan artefak, metadata, status, serta metrik yang dapat dipantau. Nilai numerik final perlu diisi berdasarkan dataset eksperimen yang disepakati.

Draft ini: jumlah halaman dan lampiran disesuaikan setelah finalisasi.

ABSTRACT

MUHAMMAD RAYHAN ARLIANSYAH. 51422133.

IMPLEMENTATION OF AN ML SERVICE COMPONENT AND LLM ANALYSIS IN A WEB-BASED AUTOMATED MACHINE LEARNING PLATFORM. Undergraduate Thesis. Your Study Program, Faculty of Computer Science and Information Technology, Gunadarma University, 2026.

Keywords: ML Service, Automated Machine Learning, preprocessing, Scikit-learn, Large Language Model, Ollama, ROUGE.

This study implements an ML Service component and an LLM analysis pipeline in a web-based Automated Machine Learning platform. The ML Service supports three preprocessing paths: tabular data using ColumnTransformer, imputation, scaling, one-hot encoding, and LabelEncoder; image data using resizing and Histogram of Oriented Gradients; and text data using cleaning, optional stopwords removal, and TF-IDF. The training module provides algorithm selection, data splitting, classification or regression metrics, and model packaging together with preprocessing artifacts. The system also integrates local large language model analysis through Ollama using few-shot prompting, output parsing, generation time, average output length, and ROUGE. Long-running jobs are executed using FastAPI background tasks so that the API can return a job identifier early. The implementation produces modularly callable pipelines, artifacts, metadata, status, and metrics. Final numerical values must be filled using the agreed experimental datasets.

KATA PENGANTAR

Dengan mengucapkan segala puji dan syukur atas kehadiran Allah SWT, penulis dapat menyelesaikan penyusunan skripsi ini. Skripsi ini disusun sebagai salah satu syarat untuk menyelesaikan Program Sarjana Strata Satu di Universitas Gunadarma.

Skripsi ini mendokumentasikan kontribusi penulis sebagai ML Engineer pada proyek AutoML Platform v2. Fokus pembahasan diarahkan pada preprocessing multi-tipe data, pengelolaan eksperimen dan training, pengemasan artefak model, serta integrasi analisis teks berbasis LLM lokal. Komponen backend dan frontend yang dikembangkan anggota tim lain dibahas sejauh diperlukan untuk menjelaskan integrasi.

Penulis menyampaikan terima kasih kepada dosen pembimbing, seluruh anggota tim, dan semua pihak yang memberikan dukungan selama penelitian. Penulis menyadari bahwa skripsi ini masih dapat dikembangkan. Kritik dan saran yang membangun sangat diharapkan.

KOTA, 2026

MUHAMMAD RAYHAN ARLIANSYAH

DAFTAR ISI

HALAMAN JUDUL	i
PERNYATAAN ORISINALITAS DAN PUBLIKASI	ii
LEMBAR PENGESAHAN	iii
ABSTRAK	iv
ABSTRACT	v
KATA PENGANTAR	vi
DAFTAR ISI	vii
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
DAFTAR LAMPIRAN	xii
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Ruang Lingkup	3
1.4 Tujuan Penelitian	4
1.5 Kontribusi Penelitian	5
1.6 Sistematika Penulisan	5
2 TINJAUAN PUSTAKA	8
2.1 Machine Learning dan Automated Machine Learning	8
2.1.1 Machine Learning Service	8
2.1.2 Automated Machine Learning	9

2.1.3	Python dan Scikit-learn	10
2.2	Preprocessing Data	10
2.2.1	Preprocessing Tabular	11
2.2.2	Preprocessing Gambar	12
2.2.3	Preprocessing Teks	14
2.3	Algoritma dan Evaluasi Model	15
2.3.1	Algoritma Pembelajaran	15
2.3.2	Evaluasi Model	17
2.4	Large Language Model	18
2.4.1	Large Language Model dan Ollama	19
2.4.2	Few-shot Prompting dan Analisis Kualitatif	20
2.4.3	ROUGE	21
2.5	Kerangka Kerja Metodologis	23
2.5.1	Design Science Research	23
2.5.2	Metodologi Rekayasa Pemodelan CRISP-DM	24
2.5.3	Privacy by Design dan Keamanan Data	25
2.6	Penelitian Terkait	25
3	METODE PENELITIAN	28
3.1	Pendekatan Design Science Research	28
3.2	Metodologi Rekayasa Pemodelan CRISP-DM	30
3.3	Batas Kontribusi Tim	32
3.4	Arsitektur Sistem	33
3.5	Rancangan Pipeline Preprocessing	33
3.6	Rancangan Eksperimen dan Training	34
3.7	Rancangan Analisis LLM	35
3.8	Rancangan Privacy Gate	36
3.9	Pemilihan Teknologi dan Pengujian	37
4	HASIL DAN PEMBAHASAN	38
4.1	Lingkungan Implementasi	38
4.2	Implementasi Preprocessing Tabular	38
4.3	Implementasi Preprocessing Gambar	39
4.4	Implementasi Preprocessing Teks	39
4.5	Implementasi Training dan Evaluasi	40
4.6	Integrasi Background Task	40

4.7 Implementasi Analisis LLM	40
4.8 Pengujian Preprocessing	41
4.9 Pengujian Eksperimen dan Training	41
4.10 Pengujian Analisis LLM	42
4.11 Pembahasan dan Keterbatasan	42
5 PENUTUP	44
5.1 Kesimpulan	44
5.2 Saran	44
DAFTAR PUSTAKA	46
LAMPIRAN	1

DAFTAR TABEL

2.1	Ringkasan Penelitian Terkait	27
3.1	Implementasi Tahapan Design Science Research (DSR) pada Proyek AutoML Platform v2	30
3.2	Pemetaan Fasa Iteratif CRISP-DM terhadap Modul Operasional Komponen ML Service	32
3.3	Pembagian Komponen	33
3.4	Konfigurasi dan Artefak	34
3.5	Matriks Algoritma	35
3.6	Teknologi	37
4.1	Berkas Fokus	38
4.2	Transisi Status	40
4.3	Pengujian Preprocessing	41
4.4	Pengujian Training	41
4.5	Pengujian LLM	42

DAFTAR GAMBAR

2.1	Ilustrasi Pembagian Sel Spasial dan Normalisasi Blok pada Ekstraksi Fitur HOG (Dalal & Triggs, 2005)	13
2.2	Arsitektur Mekanisme Perhatian Diri (Scaled Dot-Product dan Multi-Head Attention) pada Transformer (Vaswani et al., 2017)	19
2.3	Siklus Hidup dan Fasa Iteratif Metodologi Rekayasa Pemodelan CRISP-DM (Shearer, 2000)	24
3.1	Arsitektur komponen ML Engineer pada AutoML Platform v2	33
3.2	Pipeline preprocessing tiga tipe data	34
3.3	Alur eksperimen dan training	35
3.4	Alur analisis LLM lokal dan evaluasi ROUGE	36
3.5	Gate persetujuan sebelum dataset diproses	36

DAFTAR LAMPIRAN

Lampiran 1 Listing Program ML Service	1
Lampiran 2 Contoh Konfigurasi Eksperimen	1
Lampiran 3 Checklist Finalisasi	2

Bab 1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan pesat kecerdasan buatan dan pembelajaran mesin (*machine learning*) telah mengubah paradigma analisis data di berbagai bidang penelitian ilmiah menjadi berbasis bukti kuantitatif dan kualitatif. Namun, pengembangan model *machine learning* konvensional masih dihadapkan pada hambatan teknis yang signifikan, meliputi tuntutan penguasaan bahasa pemrograman, rekayasa fitur tabular, pengelolaan pemrosesan citra, hingga orchestrasi alur kerja eksperimen. Kesenjangan kompetensi teknis tersebut menjadi kendala utama bagi peneliti bidang non-teknis yang memiliki kekayaan data empiris tetapi terbatas dalam sumber daya rekayasa komputasi untuk membangun *pipeline* pemodelan dari tahap awal (Giraldo & Laso, 2025).

Platform *Automated Machine Learning* (AutoML) berkembang sebagai solusi untuk mereduksi beban rekayasa tersebut melalui otomatisasi alur pemrosesan, seleksi algoritma, pelatihan, dan evaluasi model (Feurer et al., 2015; Hutter, Kotthoff, & Vanschoren, 2019). Dalam konteks rekayasa perangkat lunak dan komputasi skala besar, pemrosesan *machine learning* dan inferensi *Large Language Model* (LLM) memiliki karakteristik komputasi intensif (*CPU-bound*, *memory-bound*, hingga *GPU-bound*). Apabila eksekusi logika pemodelan dan inferensi model disatukan dalam satu layanan monolitik dengan server web atau *backend* utama yang bersifat *I/O-bound*, beban komputasi intensif tersebut dapat memblokir eksekusi *thread* utama dan menurunkan responsivitas sistem. Oleh karena itu, diperlukan arsitektur terdesentralisasi (*decoupled microservice*) yang memisahkan komponen *ML Service* khusus dari server *backend* utama guna menjamin stabilitas eksekusi, modularitas alur kerja, dan standardisasi penyimpanan artefak eksperimen.

Tantangan arsitektural tersebut semakin kompleks dengan beragamnya modalitas data penelitian. Data tabular, citra digital, dan dokumen teks memiliki karakteristik struktural yang berbeda sehingga menuntut penerapan *pipeline preprocessing* spesifik. Data tabular memerlukan imputasi nilai hilang serta transformasi pengodean (*encoding*) dan penskalaan (*scaling*) (Pedregosa et al., 2011). Data citra menuntut penyesuaian resolusi spasial dan ekstraksi deskriptor fitur vektor berbasis *Histogram of Oriented Gradients* (HOG) (Dalal & Triggs, 2005). Sementara itu, data teks membutuhkan pembersihan naskah dan pembentukan matriks representasi numerik berbasis *Term Frequency-Inverse Document Frequency* (TF-IDF) (Salton & Buckley, 1988). Diperlukan sebuah mekanisme standarisasi kontrak keluaran yang konsisten berupa artefak numerik siap latih (*X.pkl* dan *y.pkl*) serta berkas metadata (*meta.json*) antar-modalitas data.

Selain data kuantitatif, penelitian empiris juga kerap melibatkan analisis data kualitatif seperti transkrip wawancara yang membutuhkan ekstraksi makna semantik. Integrasi *Large Language Model* (LLM) mampu mendukung otomatisasi analisis tematik dan ringkasan eksekutif (Barros et al., 2025). Namun, pemanfaatan layanan LLM komersial berbasis komputasi awan (*public cloud*) berisiko melanggar prinsip perlindungan privasi data penelitian dan Undang-Undang Pelindungan Data Pribadi (UU PDP), serta menelan biaya operasional yang tinggi berdasarkan akumulasi konsumsi token (*pay-per-token API cost*). Guna menjawab urgensi privasi, keamanan, dan efisiensi biaya tersebut, eksekusi LLM secara lokal melalui platform Ollama menjadi solusi strategis yang mengisolasi pemrosesan data sensitif di dalam lingkungan peladen mandiri (*on-premise*) tanpa biaya berlangganan atau biaya per-token, berlandaskan prinsip *Privacy by Design* dengan dukungan metode *few-shot prompting* untuk memandu kualitas keluaran model (Ollama, 2026).

Berdasarkan pertimbangan arsitektural dan urgensi metodologis tersebut, penelitian ini difokuskan pada perancangan dan implementasi komponen *ML Service* terdesentralisasi pada platform *AutoML* berbasis web (*AutoML Platform v2*). Kontribusi pada komponen *ML Service* mencakup pengembangan *pipeline preprocessing* multi-modalitas (tabular, gambar, dan teks), pengelolaan eksperimen dan pelatihan model *supervised learning* berbasis Scikit-learn, pengemasan artefak model, serta integrasi analisis kualitatif berbasis LLM lokal yang hemat biaya dan memproteksi privasi data

penelitian (kiki-kisaki, 2026).

1.2 Rumusan Masalah

Rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang arsitektur komponen *ML Service* yang terdesentralisasi (*decoupled*) dari *backend* utama guna memproses data tabular, gambar, dan teks melalui *pipeline preprocessing* terpisah dengan kontrak artefak keluaran yang terstandardisasi?
2. Bagaimana merancang dan memvalidasi alur pengelolaan eksperimen serta pelatihan model *supervised learning* berbasis Scikit-learn secara modular, asinkron, dan dapat dipantau melalui antarmuka pemrograman aplikasi (API)?
3. Bagaimana mengintegrasikan analisis semantik data penelitian kualitatif menggunakan *Large Language Model* (LLM) secara lokal pada platform Ollama berbasis *few-shot prompting* guna memitigasi risiko privasi dan biaya token API awan, serta menilai kualitas keluarannya menggunakan metrik evaluasi ROUGE?
4. Bagaimana memvalidasi fungsionalitas, keandalan, dan keterulangan eksperimen (*reproducibility*) komponen *ML Service* pada tahapan *preprocessing*, pelatihan, pengemasan model, dan analisis LLM melalui serangkaian skenario pengujian sistematis?

1.3 Ruang Lingkup

Ruang lingkup dalam penelitian ini dibatasi pada aspek-aspek berikut:

1. Kontribusi penelitian difokuskan pada perancangan dan implementasi komponen *ML Service* sebagai bagian dari arsitektur platform *AutoML Platform v2*, sedangkan pengembangan *frontend* berbasis web dan manajemen otentikasi *backend* merupakan kontribusi anggota tim lain.
2. Modalitas data yang didukung pada *pipeline preprocessing* terbatas pada berkas tabular berformat CSV, kumpulan citra digital di dalam arsip ZIP dengan struktur subfolder kelas, dan berkas teks berformat TXT.

3. *Pipeline preprocessing* data tabular mendukung penanganan nilai hilang (*imputation*), transformasi *StandardScaler* dan *MinMaxScaler*, serta pengodean variabel kategorikal (*OneHotEncoder* dan *LabelEncoder*).
4. *Pipeline preprocessing* data gambar difokuskan pada standardisasi dimensi (*image resize*) dan ekstraksi fitur *Histogram of Oriented Gradients* (HOG) tanpa melibatkan pemodelan *deep learning* atau *transfer learning*.
5. *Pipeline preprocessing* data teks menerapkan pembersihan karakter (*text cleaning*), penyaringan kata hubung (*stopword removal*), dan ekstraksi representasi vektor *Term Frequency-Inverse Document Frequency* (TF-IDF).
6. Pelatihan model difokuskan pada algoritma *supervised learning* klasifikasi dan regresi dari pustaka Scikit-learn dengan skema pembagian dataset 80 persen latih dan 20 persen uji menggunakan parameter statis acak guna menjamin keterulangan eksperimen (*reproducibility*).
7. Analisis LLM dijalankan secara lokal menggunakan platform Ollama pada lingkungan peladen mandiri berlandaskan prinsip *Privacy by Design* serta efisiensi biaya tanpa ketergantungan token API eksternal, dengan evaluasi kuantitatif kesamaan naskah berbasis metrik ROUGE.

1.4 Tujuan Penelitian

Tujuan yang dicapai dalam penelitian ini adalah sebagai berikut:

1. Merancang dan mengimplementasikan komponen *ML Service* terdesentralisasi yang mampu memproses data tabular, gambar, dan teks melalui *pipeline preprocessing* dengan kontrak keluaran artefak numerik (*X.pkl*, *y.pkl*) dan metadata (*meta.json*) yang konsisten dan terstandardisasi.
2. Merancang dan mengimplementasikan modul pengelolaan eksperimen dan pelatihan model *supervised learning* Scikit-learn yang mampu memvalidasi konfigurasi, melatih estimator, menghitung metrik performa, serta mengemas artefak model.

3. Mengintegrasikan layanan analisis teks lokal menggunakan *Large Language Model* (LLM) pada platform Ollama berbasis *few-shot prompting* yang memproteksi privasi data dan mengeliminasi biaya token, serta menghitung metrik evaluasi ROUGE.
4. Melakukan verifikasi dan pengujian fungsionalitas secara menyeluruh pada alur *preprocessing*, pelatihan model, dan analisis LLM guna menjamin keandalan dan keterulangan (*reproducibility*) komponen *ML Service*.

1.5 Kontribusi Penelitian

Penelitian ini memberikan kontribusi metodologis dan arsitektural melalui rancangan pemisahan beban komputasi intensif (*decoupled architecture*) antara komponen *ML Service* dan server *backend* utama pada platform *AutoML* berbasis web. Rancangan ini memperkenalkan kontrak standardisasi artefak data lintas-modalitas (tabular, citra, dan teks) yang menyeragamkan keluaran *preprocessing* ke dalam matriks fitur terstruktur ($X.pkl$, $y.pkl$) dan deskriptor eksperimen ($meta.json$), sehingga memungkinkan isolasi eksekusi alur kerja pemodelan secara modular tanpa mengganggu kinerja respons layanan utama.

Secara praktis dan teknis, penelitian ini menyediakan implementasi nyata komponen *ML Service* untuk *AutoML Platform v2* yang memungkinkan peneliti non-teknis menjalankan alur rekayasa data, seleksi algoritma Scikit-learn, serta analisis kualitatif secara otomatis. Selain itu, integrasi inferensi *Large Language Model* (LLM) lokal menggunakan platform Ollama memberikan solusi praktis analisis data tekstual berlandaskan *Privacy by Design* sekaligus efisiensi anggaran penelitian, yang melindungi kedaulatan data sensitif dan mengeliminasi biaya akumulatif token pada layanan komputasi awan publik.

1.6 Sistematika Penulisan

Sistematika penulisan dalam skripsi ini disusun berdasarkan sistematika berikut:

1. Bab 1 Pendahuluan memaparkan latar belakang masalah yang menguraikan urgensi pemisahan arsitektur *ML Service* pada platform *AutoML* dan pentingnya eksekusi *Large Language Model* (LLM) lokal berdasarkan privasi data serta efisiensi biaya. Bab ini juga merumuskan pertanyaan penelitian, menetapkan ruang lingkup dan batasan sistem, merumuskan tujuan yang dicapai, mengklasifikasikan kontribusi metodologis dan praktis, serta menjelaskan sistematika penulisan dokumen secara utuh.
2. Bab 2 Tinjauan Pustaka menguraikan landasan teori yang relevan dan tinjauan literatur terkini yang mendasari pengembangan sistem. Pembahasan mencakup konsep arsitektur *microservice* dalam *machine learning*, prinsip *Automated Machine Learning* (AutoML), teknik *preprocessing* data tabular, citra, dan teks, algoritma *supervised learning* beserta metrik evaluasi Scikit-learn, arsitektur *Large Language Model* (LLM) dan eksekusi lokal via Ollama, metrik evaluasi ROUGE, kerangka kerja *Design Science Research* (DSR), metodologi Waterfall, serta kajian perbandingan terhadap penelitian terkait.
3. Bab 3 Metode Penelitian menguraikan metodologi dan rancangan teknis pengembangan komponen *ML Service* menggunakan pendekatan *Design Science Research* (DSR). Bab ini merinci tahapan pengembangan, pembagian tugas antar-kontributor proyek, rancangan arsitektur sistem, desain alur *pipeline preprocessing* untuk tiga modalitas data, rancangan pengelolaan eksperimen dan pelatihan model, rancangan integrasi LLM berbasis *few-shot prompting*, arsitektur keamanan *Privacy by Design*, serta skema pengujian sistematis.
4. Bab 4 Hasil dan Pembahasan menyajikan hasil implementasi rancangan komponen *ML Service* beserta evaluasi kualitatif dan kuantitatif terhadap kinerjanya. Bab ini memaparkan realisasi kode pada modul *preprocessing* tabular, gambar, dan teks, implementasi alur pelatihan serta pengemasan model Scikit-learn, eksekusi analisis LLM lokal Ollama, hasil pengujian fungsionalitas dan performa seluruh modul, serta diskusi mendalam mengenai keterbatasan implementasi yang ditemukan.
5. Bab 5 Penutup menyarikan kesimpulan akhir yang menjawab rumusan masalah dan tujuan penelitian berdasarkan bukti pengujian empiris.

Bab ini juga merumuskan saran penelitian lanjutan yang murni bersifat ilmiah dan metodologis untuk menjadi landasan pengembangan keilmuan bagi peneliti berikutnya.

Bab 2. TINJAUAN PUSTAKA

2.1 Machine Learning dan Automated Machine Learning

Bagian ini memaparkan landasan teori dan tinjauan literatur mengenai konsep *machine learning* dan *Automated Machine Learning* (AutoML) yang menjadi dasar pengembangan arsitektur komponen *ML Service*. Pembahasan mencakup paradigma arsitektur *microservice* dalam pemrosesan *machine learning*, kerangka teoritis *AutoML*, serta ekosistem pustaka bahasa pemrograman Python dan Scikit-learn yang mendasari implementasi sistem.

2.1.1 Machine Learning Service

Dalam rekayasa perangkat lunak modern untuk sistem analitik data, *ML Service* merupakan sebuah komponen arsitektur layanan terdesentralisasi (*microservice*) yang dirancang khusus untuk mengeksekusi beban kerja pembelajaran mesin (*machine learning workloads*). Berbeda dengan layanan web konvensional yang berfokus pada manajemen masukan-keluaran (*I/O-bound*) serta interaksi antarmuka pengguna, layanan *machine learning* memiliki karakteristik komputasi intensif berbasis utilisasi prosesor (*CPU-bound*) dan memori (*memory-bound*). Pemisahan logika pemodelan ke dalam layanan mandiri (*decoupled architecture*) memungkinkan isolasi sumber daya komputasi, penskalaan layanan secara independen, serta pencegahan kegagalan sistem utama akibat beban proses pelatihan model yang berat.

Guna memfasilitasi interoperabilitas dengan server *backend* utama, *ML Service* mengadopsi pola desain antarmuka berbasis kontrak keluaran yang deterministik dan terstandardisasi. Antarmuka komunikasi menerima spesifikasi konfigurasi eksperimen dan parameter lokasi data, kemudian mengor-

kestrasi eksekusi *pipeline preprocessing* dan pelatihan model secara asinkron. Kontrak keluaran layanan mengembalikan status eksekusi (seperti *completed* atau *failed*), laporan kesalahan deskriptif apabila terjadi anomali, metrik evaluasi model, serta lokasi direktori penyimpanan artefak numerik (*X.pkl*, *y.pkl*), model terlatih, dan berkas metadata (*meta.json*). Pola desain antarmuka estimator yang seragam ini mengadaptasi prinsip arsitektur perangkat lunak ilmiah yang memisahkan logika matematika dari lapisan protokol aplikasi (Buitinck et al., 2013).

2.1.2 Automated Machine Learning

Automated Machine Learning (AutoML) merupakan disiplin ilmu dalam kecerdasan buatan yang bertujuan mengotomatisasi tahapan-tahapan empiris dalam perancangan dan penerapan model *machine learning*. Secara teoritis, masalah utama dalam *AutoML* diformulasikan sebagai *Combined Algorithm Selection and Hyperparameter Optimization* (CASH), yaitu pencarian kombinasi optimal antara alur pra-pemrosesan data, pemilihan algoritma pembelajaran, serta penetapan hiperparameter yang memberikan kinerja terbaik pada suatu dataset (Feurer et al., 2015; Hutter et al., 2019). Otomatisasi ini bertujuan mengurangi ketergantungan pada intuisi manual analisis data, mempercepat siklus eksperimen, dan meningkatkan keterulangan bereksperimen (*reproducibility*).

Dalam spektrum implementasi sistem *AutoML*, tingkat otomatisasi dapat diterapkan pada berbagai lapisan arsitektur. Sistem pada penelitian ini difokuskan pada otomatisasi tingkat eksekusi *pipeline* dan manajemen eksperimen modular. Pengguna menetapkan spesifikasi eksperimen (modalitas data, algoritma target, dan proporsi pembagian data), sementara sistem mengotomatisasi seluruh alur pra-pemrosesan, penanganan tipe data (tabular, citra, dan teks), pencatatan status eksekusi, penghitungan metrik performa, dan pengemasan artefak model. Optimasi pencarian hiperparameter otomatis (seperti *Bayesian Optimization* atau *Grid Search*) merupakan lapisan lanjutan yang dibangun di atas arsitektur *pipeline* otomatis ini dan menjadi fokus penelitian lanjutan (Bergstra, Bardenet, Bengio, & Kégl, 2011).

2.1.3 Python dan Scikit-learn

Bahasa pemrograman Python dipilih sebagai landasan implementasi *ML Service* karena tersedianya ekosistem pustaka komputasi ilmiah dan analisis data yang sangat komprehensif. Python menyediakan keunggulan interoperabilitas melalui dukungan *binding* dengan bahasa komputasi berkinerja tinggi (seperti C dan C++), struktur penulisan yang modular, serta adopsi yang luas di kalangan akademisi dan praktisi rekayasa data (Python Software Foundation, 2026).

Pustaka Scikit-learn merupakan landasan utama dalam penerapan algoritma pembelajaran klasik dan *preprocessing* pada sistem ini. Scikit-learn dirancang dengan paradigma pemrograman berorientasi objek yang menerapkan antarmuka seragam dan konsisten berdasarkan tiga peran utama: *Estimator* yang mempelajari parameter model melalui metode *fit*, *Transformer* yang melakukan transformasi data melalui metode *transform* atau *fit_transform*, dan *Predictor* yang menghasilkan estimasi label melalui metode *predict* atau *predict_proba* (Buitinck et al., 2013; Pedregosa et al., 2011).

Antarmuka pemrograman aplikasi (API) Scikit-learn yang seragam ini memungkinkan sistem *AutoML* menerapkan pola desain pabrik (*factory pattern*). Melalui pola desain ini, komponen *ML Service* dapat menginstansiasi berbagai estimator dan *pipeline preprocessing* secara dinamis berdasarkan konfigurasi eksperimen tanpa perlu mengubah kode sumber inti untuk algoritma yang berbeda. Konsistensi desain antarmuka ini merupakan fondasi penting dalam membangun arsitektur *machine learning* yang modular, terukur, dan memiliki keterulangan eksperimen (*reproducibility*) yang tinggi.

2.2 Preprocessing Data

Pra-pemrosesan data (*data preprocessing*) merupakan tahapan fundamental dalam *pipeline* pemodelan yang bertujuan membersihkan, mentransformasikan, dan menyandikan data mentah menjadi format representasi numerik yang siap diproses oleh algoritma pembelajaran mesin. Dalam rancangan komponen *ML Service*, karakteristik struktural data yang berbeda menuntut penerapan alur transformasi spesifik. Bagian ini menguraikan landasan teoritis dan matematis dari metode *preprocessing* untuk tiga modalitas data yang didukung oleh sistem, yaitu data tabular, citra digital, dan dokumen teks.

2.2.1 Preprocessing Tabular

Data tabular berformat CSV dalam penelitian empiris kerap mengalami permasalahan ketidaklengkapan data atau nilai hilang (*missing values*), yang dapat disebabkan oleh kegagalan sensor, kesalahan entri, atau ketidaktersediaan rekor atribut. Dalam kaidah statistika dan *machine learning*, algoritma pembelajaran konvensional umumnya tidak dapat memproses matriks fitur yang mengandung nilai kosong (*null* atau *Not a Number*). Oleh karena itu, diperlukan teknik imputasi nilai hilang (*missing value imputation*). Pada sistem ini, penanganan atribut numerik menerapkan imputasi berbasis pemusatan data menggunakan nilai tengah (*median*) untuk distribusi data yang condong (*skewed*) atau rata-rata (*mean*), sedangkan untuk atribut kategorikal menerapkan imputasi nilai modus (*most frequent value*) (Pedregosa et al., 2011).

Variabel numerik dalam dataset tabular sering kali memiliki rentang skala ukur yang sangat berbeda. Perbedaan skala tersebut dapat mendistorsi fungsi objektif pada algoritma berbasis jarak (seperti *k-Nearest Neighbors* dan *Support Vector Machines*) maupun memperlambat konvergensi pada algoritma berbasis gradien. Untuk mengatasi permasalahan skala, sistem mendukung dua metode transformasi utama: *standardization* menggunakan *StandardScaler* dan *normalization* menggunakan *MinMaxScaler*. *StandardScaler* mentransformasikan setiap nilai atribut x menjadi skor baku z dengan nilai tengah nol ($\mu = 0$) dan varians satu ($\sigma^2 = 1$), yang dirumuskan pada Persamaan (2.1):

$$z = \frac{x - \mu}{\sigma}$$

di mana μ merupakan rata-rata populasi atribut dan σ merupakan simpangan baku atribut. Sementara itu, *MinMaxScaler* memetakan nilai atribut ke dalam rentang batas kontinu $[0, 1]$, yang sesuai untuk distribusi data berrentang terbatas.

Atribut yang bernilai kategorikal atau string nominal wajib dikodekan menjadi representasi angka numerik agar dapat diproses oleh operasi aljabar linier estimator. Atribut fitur kategorikal mentransformasikan variabel nominal menjadi vektor biner ortogonal menggunakan metode *OneHotEncoder*, di mana setiap kategori unik dibentuk menjadi atribut variabel biner

mandiri guna mencegah pembentukan hubungan urutan (*ordinality*) semu antar-kategori. Untuk kolom target (*target column*) pada tugas klasifikasi, variabel label disandikan ke dalam bilangan bulat $0, 1, \dots, K - 1$ menggunakan *LabelEncoder*.

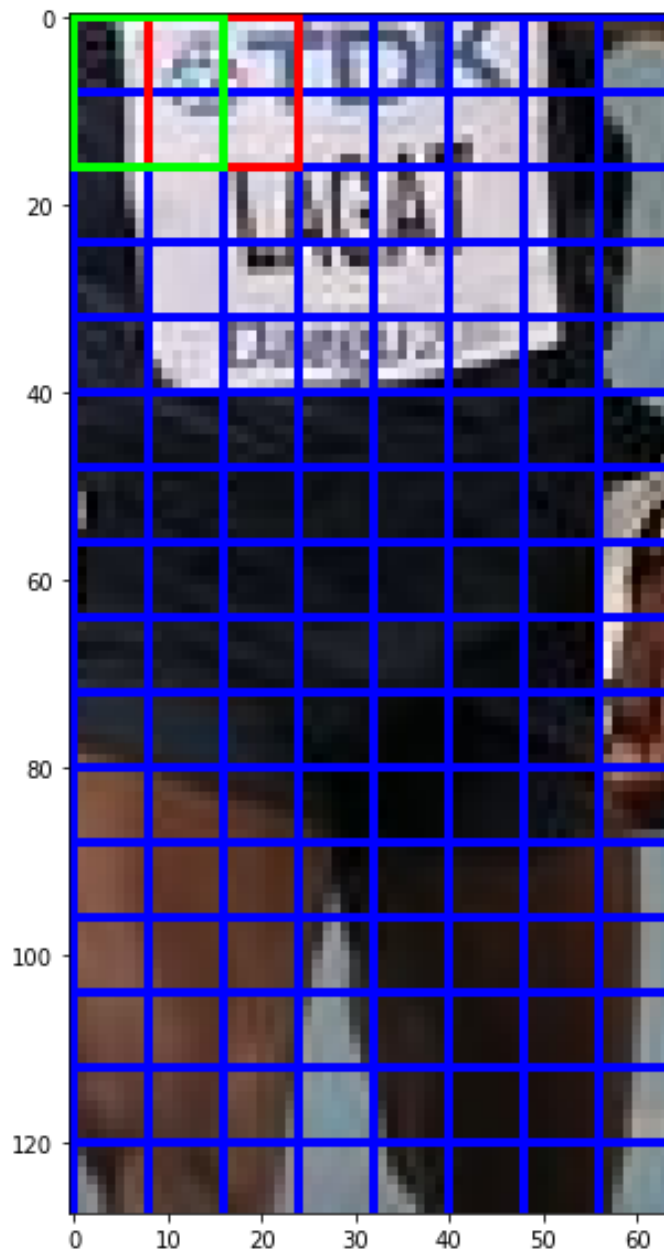
Guna menjaga konsistensi eksekusi alur pemrosesan, transformasi atribut numerik dan kategorikal digabungkan ke dalam satu antarmuka terpadu menggunakan kelas *ColumnTransformer* (Buitinck et al., 2013). Seluruh objek penata transformasi yang telah mempelajari parameter dari data latih (*fitting*) disimpan secara persisten ke dalam berkas artefak serialisasi (*preprocessor.pkl*). Penyimpanan artefak transformasi ini sangat esensial guna memvalidasi bahwa transformasi yang identik akan diterapkan pada data uji maupun data baru saat inferensi, sehingga memitigasi risiko kebocoran data (*data leakage*) dan menjamin keterulangan eksperimen (*reproducibility*).

2.2.2 Preprocessing Gambar

Kumpulan citra digital di dalam arsip ZIP pada tahap ingest data kerap memiliki resolusi spasial dan ruang warna (*color space*) yang heterogen. Algoritma pembelajaran mesin konvensional membutuhkan matriks masukan berdimensi seragam. Oleh karena itu, langkah awal pada *pipeline preprocessing* citra mencakup standardisasi dimensi spasial melalui penyesuaian resolusi (*image resize*) ke dimensi piksel tetap, serta konversi format warna dari *Red-Green-Blue* (RGB) ke format derajat keabuan (*grayscale*) untuk mereduksi redundansi intensitas warna yang tidak signifikan terhadap deteksi bentuk objek.

Guna menghasilkan vektor representasi berdimensi rendah yang tangguh terhadap variasi pencahayaan dan pergeseran spasial minor, sistem menerapkan metode ekstraksi fitur *Histogram of Oriented Gradients* (HOG) (Dalal & Triggs, 2005). Secara matematis, ekstraksi HOG menghitung nilai gradien horizontal (G_x) dan gradien vertikal (G_y) pada setiap piksel (x, y) menggunakan kernel derivatif diferensial. Nilai magnitudo gradien (m) dan orientasi sudut (θ) dirumuskan pada Persamaan (2.2) dan Persamaan (2.3):

Mekanisme pembagian sel spasial, penghitungan histogram orientasi gradien lokal, dan normalisasi blok beririsan ini diilustrasikan pada Gambar 2.1.



Gambar 2.1: Ilustrasi Pembagian Sel Spasial dan Normalisasi Blok pada Eks-traksi Fitur HOG (Dalal & Triggs, 2005)

$$m(x, y) = \sqrt{G_x^2 + G_y^2}$$

$$\theta(x, y) = \arctan \left(\frac{G_y}{G_x} \right)$$

Piksel-piksel selanjutnya dikelompokkan ke dalam sel spasial (misalnya ukuran 8×8 piksel) untuk menyusun histogram orientasi lokal. Histogram pada sel yang berdekatan kemudian dikelompokkan ke dalam blok spasial dan dinormalisasi secara kontras (menggunakan norma L2 atau *L2-hys*) guna mengeliminasi efek iluminasi lokal (Dalal & Triggs, 2005). Deskriptor vektor fitur numerik HOG yang dihasilkan menjadi representasi padat yang sangat efektif untuk melatih estimator klasifikasi klasik (seperti *Support Vector Machines* atau *Random Forest*) pada platform *AutoML* tanpa memerlukan sumber daya komputasi tinggi dari arsitektur *deep learning*.

2.2.3 Preprocessing Teks

Dokumen teks mentah mengandung berbagai deret karakter non-informatif yang berpotensi menurunkan kualitas pemodelan semantik maupun klasifikasi naskah. Tahapan awal pemrosesan teks menerapkan normalisasi berurutan yang mencakup penyeragaman huruf menjadi huruf kecil (*case folding*), penghapusan tautan *Uniform Resource Locator* (URL), penyaringan karakter tanda baca dan non-alfanumerik, serta perapihan spasi berlebih. Selain itu, sistem mendukung penyaringan kata hubung umum (*stopword removal*) secara opsional untuk mengeliminasi kata-kata bernilai frekuensi tinggi yang kurang mengandung ciri diskriminatif antar-kelas dokumen (Kowsari et al., 2019).

Dokumen yang telah dibersihkan ditransformasikan ke dalam ruang vektor numerik menggunakan metode pembobotan *Term Frequency-Inverse Document Frequency* (TF-IDF). Berbeda dengan representasi frekuensi kata sederhana (*Bag-of-Words*), metode TF-IDF memberikan bobot tinggi pada istilah yang muncul sering dalam satu dokumen tertentu namun jarang muncul secara global di seluruh korpus dokumen (Salton & Buckley, 1988). Secara matematis, bobot TF-IDF untuk suatu istilah t pada dokumen d dalam korpus D dirumuskan pada Persamaan (2.4):

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

di mana komponen *Inverse Document Frequency* (IDF) dihitung dengan Persamaan (2.5):

$$\text{IDF}(t, D) = \log \left(\frac{1 + |D|}{1 + |\{d \in D : t \in d\}|} \right) + 1$$

Melalui perkalian bobot ini, istilah khusus yang menjadi ciri penanda suatu topik dokumen akan memiliki bobot vektor yang dominan, sehingga menghasilkan matriks numerik jarang (*sparse matrix*) yang sangat diskriminatif untuk dilatih oleh estimator *supervised learning* (Kowsari et al., 2019; Salton & Buckley, 1988).

2.3 Algoritma dan Evaluasi Model

Bagian ini menguraikan landasan teori dan formulasi matematis dari algoritma pembelajaran terawasi (*supervised learning*) yang digunakan dalam modul pelatihan sistem, serta definisi matematis dari metrik evaluasi kinerja yang dihitung untuk tugas klasifikasi dan regresi. Pemilihan lima keluarga algoritma pembelajaran ini dilandasi oleh representasi kerangka kerja yang beragam, mencakup model pohon keputusan, model gabungan (*ensemble learning*), model berbasis margin optime, model non-parametrik berbasis jarak, dan model linier probabilitatik.

2.3.1 Algoritma Pembelajaran

Pohon Keputusan (*Decision Tree*) merupakan metode pembelajaran non-parametrik yang membentuk aturan keputusan hierarkis melalui partisi rekursif ruang fitur berdasarkan ukuran ketidakmurnian kelas (*class impurity*) (Quinlan, 1986). Dalam induksi pohon keputusan klasifikasi, kriteria pemisahan simpul umumnya dievaluasi menggunakan indeks *Gini Impurity* (I_G). Secara matematis, nilai *Gini Impurity* pada suatu simpul t untuk dataset dengan K kelas dirumuskan pada Persamaan (2.6):

$$I_G(t) = 1 - \sum_{k=1}^K p_k^2$$

di mana p_k adalah proporsi sampel yang memiliki kelas k pada simpul t . Algoritma akan memilih atribut fitur yang menghasilkan penurunan ketidakmurnian terbesar pada setiap cabang partisi.

Hutan Acak (*Random Forest*) merupakan pengembangan dari pohon keputusan berlandaskan paradigma pembelajaran gabungan (*ensemble learning*) yang mengombinasikan teknik agregasi bootstrap (*bootstrap aggregating* atau *bagging*) dengan pemilihan sub-ruang fitur acak pada setiap pemisahan simpul (Breiman, 2001). Dengan melatih banyak pohon keputusan secara independen pada sampel bootstrap yang berbeda dan mengambil keputusan akhir melalui pemungutan suara terbanyak (*majority voting*) untuk klasifikasi atau rata-rata untuk regresi, *Random Forest* mampu mereduksi varians model, memitigasi risiko *overfitting*, serta memberikan stabilitas performa yang tinggi pada data tabular, vektor fitur HOG, maupun matriks TF-IDF.

Mesin Vektor Pendukung (*Support Vector Machines* atau SVM) merupakan metode pembelajaran berbasis margin yang mencari hiperbidang pemisah optimal (*maximum margin hyperplane*) antar-kelas dalam ruang fitur berdimensi tinggi (Cortes & Vapnik, 1995). Untuk tugas regresi, varian algoritma ini diterapkan sebagai *Support Vector Regression* (SVR) yang mengoptimalkan fungsi objektif dalam batas margin toleransi ϵ . Secara matematis, formulasi optimasi primal untuk SVM linier dengan variabel *slack* ξ_i dirumuskan pada Persamaan (2.7):

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

dengan kendala $y_i(w \cdot \phi(x_i) + b) \geq 1 - \xi_i$ dan $\xi_i \geq 0$, di mana w adalah vektor bobot normal, C adalah parameter regularisasi yang mengendalikan trade-off antara margin batas dan kesalahan klasifikasi, serta $\phi(x)$ adalah fungsi pemetaan kernel ke ruang berdimensi lebih tinggi.

k-Tetangga Terdekat (*k-Nearest Neighbors* atau k-NN) merupakan metode pembelajaran berbasis sampel non-parametrik yang memprediksi label instans baru berdasarkan kesamaan jarak spasial terhadap k sampel tetangga terdekat di dalam ruang fitur pelatihan (Cover & Hart, 1967). Perhitungan kedekatan sampel umumnya diterapkan menggunakan metrik jarak Euclidean (*Euclidean distance*) $d(x, x')$ antar-vektor fitur x dan x' berdimensi p , yang dirumuskan pada Persamaan (2.8):

$$d(x, x') = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}$$

Regresi Logistik (*Logistic Regression*) merupakan model probabilistik linier yang memodelkan posterior probabilitas keanggotaan kelas klasifikasi biner maupun multikelas dengan memanfaatkan transformasi fungsi penghubung sigmoid (*sigmoid link function*). Secara matematis, estimasi probabilitas keanggotaan kelas positif $P(y = 1|x)$ untuk vektor fitur masukan x dirumuskan pada Persamaan (2.9):

$$P(y = 1|x) = \frac{1}{1 + e^{-(w \cdot x + b)}}$$

di mana w adalah parameter bobot fitur dan b adalah bias pemotong. Efisiensi komputasi serta kemampuan penafsiran parameter yang tinggi menjadikan regresi logistik sebagai estimator dasar (*baseline estimator*) yang esensial dalam pengujian alur *AutoML* (Pedregosa et al., 2011).

2.3.2 Evaluasi Model

Guna memvalidasi efektivitas dan kualitas generalisasi model pembelajaran terawasi, modul pelatihan menghitung sekumpulan metrik evaluasi standar secara otomatis yang dirujuk dari pustaka Scikit-learn (Pedregosa et al., 2011). Evaluasi tugas klasifikasi dilandasi oleh analisis komponen matriks kebingungan (*Confusion Matrix*) yang terdiri dari nilai *True Positives* (TP), *True Negatives* (TN), *False Positives* (FP), dan *False Negatives* (FN). Empat metrik evaluasi klasifikasi utama dirumuskan pada Persamaan (2.10) hingga Persamaan (2.13):

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Pada klasifikasi multikelas atau dataset dengan distribusi kelas yang tidak seimbang (*imbalanced dataset*), sistem menghitung nilai presisi, recall, dan skor F1 berdasarkan rata-rata berbobot (*weighted average*) sesuai proporsi jumlah sampel pada masing-masing kelas aktual.

Untuk tugas regresi yang memprediksi variabel target kontinu atas n sampel pengujian dengan nilai aktual y_i dan nilai prediksi $\hat{y}_i$, sistem menghitung empat metrik kesalahan dan determinasi yang dirumuskan pada Persamaan (2.14) hingga Persamaan (2.17):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

di mana $\bar{y}$ merupakan rata-rata dari keseluruhan nilai target aktual observasi. Seluruh hasil perhitungan metrik klasifikasi maupun regresi ini disandikan secara otomatis ke dalam struktur data berkas metadata (`meta.json`) pada field `metrics`. Standar serialisasi metrik ini memungkinkan modul pemantauan bereksperimen pada platform *AutoML Platform v2* memvalidasi kinerja estimator dan mendukung perbandingan kuantitatif antar-eksperimen secara obyektif (Pedregosa et al., 2011).

2.4 Large Language Model

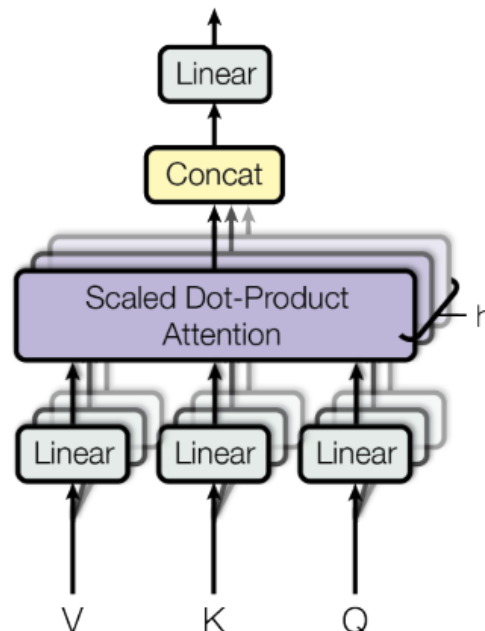
Bagian ini memaparkan landasan teori mengenai *Large Language Model* (LLM) dan arsitektur pemrosesan bahasa alami yang mendasari otomatisasi analisis teks kualitatif pada komponen *ML Service*. Pembahasan mencakup mekanisme inferensi lokal melalui platform Ollama berlandaskan pri-

vasi data dan efisiensi biaya, kerangka kerja pembelajaran dalam konteks (*In-Context Learning*) melalui *few-shot prompting*, serta definisi matematis dari keluarga metrik evaluasi ROUGE yang digunakan untuk mengukur kemiripan naskah secara kuantitatif.

2.4.1 Large Language Model dan Ollama

Large Language Model (LLM) merupakan arsitektur pemodelan bahasa berskala besar yang dibangun di atas paradigma jaringan saraf *Transformer* dan dilatih pada korpus teks masif guna mempelajari representasi semantik serta sintaksis bahasa manusia (Brown et al., 2020). Mekanisme inti yang memungkinkan LLM menangkap hubungan konteks jarak jauh antar-token dalam suatu urutan masukan adalah mekanisme perhatian diri (*self-attention mechanism*). Secara matematis, operasi perhatian perkalian titik berskala (*Scaled Dot-Product Attention*) untuk matriks *Query* (Q), *Key* (K), dan *Value* (V) dirumuskan pada Persamaan (2.18):

Arsitektur perkalian matriks *Query*, *Key*, dan *Value* beserta fungsi pemetaan *softmax* pada mekanisme perhatian diri ini diilustrasikan pada Gambar 2.2.



Gambar 2.2: Arsitektur Mekanisme Perhatian Diri (Scaled Dot-Product dan Multi-Head Attention) pada Transformer (Vaswani et al., 2017)

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

di mana d_k merepresentasikan dimensi vektor kunci (*key dimension*) yang berfungsi sebagai faktor penskalaan untuk menjaga stabilitas gradien selama eksekusi.

Dalam penerapan pada platform *AutoML Platform v2*, eksekusi inferensi LLM dijalankan secara lokal menggunakan platform Ollama sebagai peladen waktu jalan (*runtime*) (Ollama, 2026). Server *backend* mengirimkan nama model (`model_name`) beserta parameter instruksi (`prompt`) melalui protokol HTTP ke **endpoint** API inferensi. Ollama mengadopsi teknik kuantisasi model (*model quantization* berformat GGUF) yang memampatkan bobot parameter berpresisi tinggi (16-bit) menjadi representasi bilangan bulat 4-bit atau 8-bit, sehingga memungkinkan model parameter besar (seperti `mistral:7b`, `llama3.2:3b`, atau `qwen2.5:7b`) dieksekusi secara efisien pada komputasi peladen mandiri (*on-premise*) yang bersifat *CPU-bound* dan *GPU-bound* (*VRAM-bound*).

Pemilihan eksekusi LLM secara lokal via Ollama dilandasi oleh dua justifikasi akademis dan strategis yang krusial. Pertama, perlindungan privasi data penelitian berlandaskan asas *Privacy by Design* dan Undang-Undang Pelindungan Data Pribadi (UU PDP). Dokumen penelitian kualitatif seperti transkrip wawancara sering kali mengandung informasi sensitif yang tidak boleh diproses melalui layanan komputasi awan publik (*public cloud API*). Kedua, efisiensi biaya operasional (*cost efficiency*). Layanan LLM komersial menerapkan biaya akumulatif berdasarkan konsumsi token masukan dan keluaran (*pay-per-token API cost*), yang dapat menjadi beban anggaran sangat tinggi pada pengujian berskala besar. Inferensi lokal membebaskan penelitian dari ketergantungan token berbayar sekaligus menjamin kedaulatan data penelitian (Ollama, 2026).

2.4.2 Few-shot Prompting dan Analisis Kualitatif

Salah satu keunggulan utama dari arsitektur LLM modern adalah kemampuan pembelajaran dalam konteks (*In-Context Learning* atau ICL), di mana model mampu beradaptasi untuk mengeksekusi tugas spesifik baru tanpa

memerlukan penyesuaian bobot parameter melalui pelatihan ulang (*fine-tuning*) (Brown et al., 2020). Kemampuan ICL dioptimalkan melalui metode *few-shot prompting*, yaitu penyertaan sekumpulan demonstrasi contoh masukan-keluaran terstruktur $(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)$ di dalam instruksi awal (*prompt*) guna memandu distribusi probabilitas keluaran model agar mematuhi format dan gaya analisis yang dikehendaki.

Pada modul analisis kualitatif komponen *ML Service*, dataset referensi memuat tiga atribut utama: transkrip ucapan mentah (*verbatim*), pengodean tematik (*coding*), dan uraian analisis eksekutif (*analisis*). Sistem memilih tiga baris pertama dari dataset referensi sebagai demonstrasi *3-shot* untuk memandu model dalam dua skenario penelitian (Barros et al., 2025):

1. Skenario Analisis Tematik Lanjutan (dataset *llm_new*): Setiap rekor pengujian menyediakan masukan teks *verbatim* dan *coding*, kemudian model diinstruksikan menghasilkan sintesis uraian analisis kualitatif yang mendalam.
2. Skenario Pengodean dan Analisis Sekaligus (dataset *llm_raw*): Setiap rekor pengujian hanya menyediakan masukan teks *verbatim* mentah, kemudian model diinstruksikan melakukan pengodean tema sekaligus menghasilkan uraian analisis secara runtut.

Kendati penerapan *few-shot prompting* dan segmentasi konteks terbukti efektif mereduksi halusinasi teks (*hallucination*) (Shuster, Poff, Chen, Kiela, & Weston, 2021), eksekusi LLM tetap diposisikan sebagai alat bantu analisis kualitatif (*analytical decision-support tool*). Kualitas semantik serta validitas psikologis atau keilmuan dari keluaran LLM wajib ditinjau dan divalidasi oleh pakar domain (*expert judgment*) dalam kerangka evaluasi manusia (*human-in-the-loop evaluation*) guna menjamin ketepatan makna dan keterulangan eksperimen (*reproducibility*).

2.4.3 ROUGE

Guna memberikan evaluasi kuantitatif yang obyektif terhadap kualitas teks keluaran analisis LLM dibandingkan dengan referensi standar emas yang disusun ahli, sistem menerapkan keluarga metrik ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*) (Lin, 2004). Metrik ROUGE mengevaluasi tingkat kemiripan tekstual berdasarkan persentase tumpang tindih (*overlap*)

n-gram dan urutan karakter antar-naskah. Implementasi pada komponen *ML Service* menghitung tiga metrik utama: ROUGE-1 (tumpang tindih unigram/kata tunggal), ROUGE-2 (tumpang tindih bigram/dua kata berurutan), dan ROUGE-L (urutan berurutan terpanjang).

Secara matematis, perhitungan nilai recall ($R_{\text{ROUGE-N}}$) dan presisi ($P_{\text{ROUGE-N}}$) untuk metrik ROUGE-N dirumuskan pada Persamaan (2.19) dan Persamaan (2.20):

$$R_{\text{ROUGE-N}} = \frac{\sum_{\text{ref} \in \text{Ref}} \sum_{\text{gram}_n \in \text{ref}} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{\text{ref} \in \text{Ref}} \sum_{\text{gram}_n \in \text{ref}} \text{Count}(\text{gram}_n)}$$

$$P_{\text{ROUGE-N}} = \frac{\sum_{\text{ref} \in \text{Ref}} \sum_{\text{gram}_n \in \text{ref}} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{\text{hyp} \in \text{Hyp}} \sum_{\text{gram}_n \in \text{hyp}} \text{Count}(\text{gram}_n)}$$

di mana $\text{Count}_{\text{match}}(\text{gram}_n)$ mewakili jumlah maksimum n-gram yang sama antara naskah referensi dan hipotesis keluaran model. Sementara itu, metrik ROUGE-L mengukur kelancaran struktur sintaksis antar-kalimat berdasarkan *Longest Common Subsequence* (LCS) antara urutan referensi X sepanjang m dan urutan keluaran Y sepanjang n . Skor F-measure untuk ROUGE-L (F_{LCS}) dirumuskan pada Persamaan (2.21) hingga Persamaan (2.23):

$$R_{\text{LCS}} = \frac{\text{LCS}(X, Y)}{m}, \quad P_{\text{LCS}} = \frac{\text{LCS}(X, Y)}{n}$$

$$\text{ROUGE-L} = \frac{(1 + \beta^2) R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + \beta^2 P_{\text{LCS}}}$$

di mana $\beta = P_{\text{LCS}}/R_{\text{LCS}}$ merupakan parameter penimbang relatif. Pada implementasi modul, seluruh skor akhir ROUGE-1, ROUGE-2, dan ROUGE-L dinyatakan sebagai nilai F-measure (*f-measure score*) yang dibulatkan hingga empat angka desimal (Lin, 2004). Perhitungan skor diterapkan secara global dengan menggabungkan keseluruhan teks referensi dan teks hipotesis pada eksperimen yang bersangkutan. Harus dipahami secara ilmiah bahwa ROUGE merupakan metrik evaluasi tumpang tindih leksikal (*lexical overlap*) yang mengukur kemiripan tekstual, namun tidak cukup untuk menjamin ketepatan makna semantik maupun validitas psikologis naskah secara sepihak,

sehingga tetap memerlukan tinjauan keahlian manusia.

2.5 Kerangka Kerja Metodologis

Bagian ini menguraikan landasan teori mengenai kerangka kerja metodologis yang menjadi rujukan dalam perancangan penelitian ilmiah serta siklus hidup pemodelan perangkat lunak analitik data. Pembahasan mencakup kerangka teoritis *Design Science Research* (DSR), perbandingan epistemologis antara model siklus hidup linear dan metodologi iteratif *Cross-Industry Standard Process for Data Mining* (CRISP-DM), serta prinsip *Privacy by Design* sebagai standar arsitektur pelindungan data.

2.5.1 Design Science Research

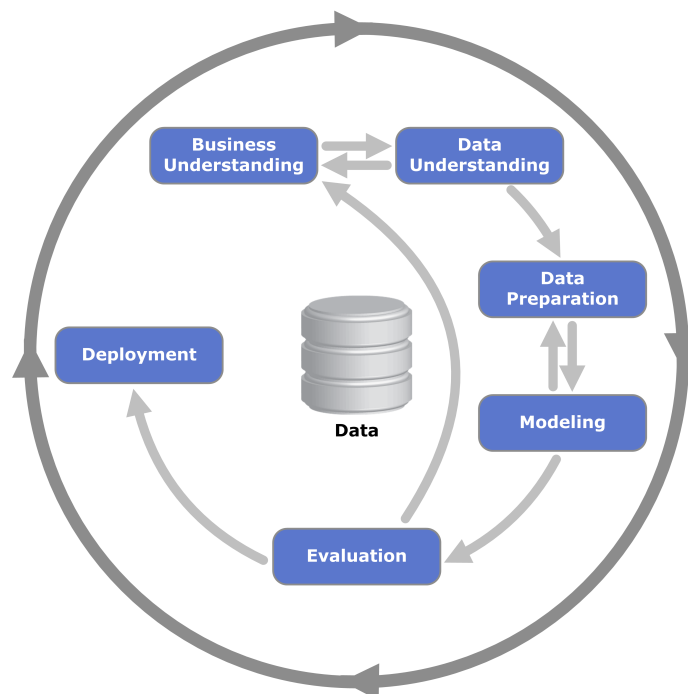
Dalam disiplin ilmu teknik informatika dan rekayasa perangkat lunak sains data, penelitian ilmiah tidak sekadar bertujuan mengamati atau memvalidasi fenomena alamiah, melainkan berfokus pada penciptaan, perancangan, dan evaluasi artefak inovatif untuk memecahkan permasalahan empiris yang relevan. Berlandaskan paradigma tersebut, kerangka kerja *Design Science Research* (DSR) dikembangkan sebagai metodologi formal yang menghubungkan teori ilmu komputer dengan rekayasa artefak praktis (Peppers, Tuunanen, Rothenberger, & Chatterjee, 2007).

Secara teoritis, metodologi DSR yang dirumuskan oleh Peppers et al. (2007) menyediakan enam fasa investigasi ilmiah yang saling iteratif: (1) Identifikasi Masalah dan Motivasi (*Problem Identification and Motivation*), yaitu pendefinisian masalah keilmuan serta justifikasi nilai dari solusi yang diusulkan; (2) Penetapan Tujuan Solusi (*Define Objectives for a Solution*), yaitu perumusan kriteria kelayakan dan tujuan kuantitatif/kualitatif dari artefak; (3) Perancangan dan Pengembangan (*Design and Development*), yaitu penciptaan konstruksi, model, atau komponen perangkat lunak; (4) Demonstrasi (*Demonstration*), yaitu penerapan artefak dalam skenario penyelesaian masalah nyata; (5) Evaluasi (*Evaluation*), yaitu pengujian empiris untuk mengukur seberapa baik artefak memenuhi tujuan solusi; dan (6) Komunikasi (*Communication*), yaitu diseminasi ilmiah atas kontribusi rancangan yang dihasilkan (Peppers et al., 2007). Metodologi ini menjadi pijakan keilmuan bagi perancangan arsitektur sistem pada bab metode penelitian.

2.5.2 Metodologi Rekayasa Pemodelan CRISP-DM

Dalam rekayasa perangkat lunak konvensional, model siklus hidup linear seperti *Waterfall* atau *Systems Development Life Cycle (SDLC)* sering diterapkan karena spesifikasi kebutuhan sistem bersifat statis dan deterministik. Namun, dalam ranah rekayasa pembelajaran mesin (*Machine Learning Engineering*), penerapan model linear memiliki keterbatasan epistemologis yang mendasar. Pembangunan model *machine learning* bersifat empiris, eksploratif, iteratif, dan sangat bergantung pada distribusi karakteristik data (*data-driven and exploratory*). Alur pemodelan tidak dapat dikunci pada fase desain statis di awal, melainkan memerlukan perputaran terus-menerus antara eksplorasi fitur, pemilihan estimator, evaluasi performa metrik, hingga perbaikan kualitas korpus data.

Berlandaskan karakteristik empiris tersebut, literatur sains data internasional mengadopsi metodologi *Cross-Industry Standard Process for Data Mining (CRISP-DM)* yang dikembangkan oleh Shearer (2000) (Shearer, 2000). Siklus hidup empiris dan hubungan interaktif antar-fasa dalam kerangka kerja CRISP-DM ini diilustrasikan pada Gambar 2.3.



Gambar 2.3: Siklus Hidup dan Fasa Iteratif Metodologi Rekayasa Pemodelan CRISP-DM (Shearer, 2000)

Secara teoritis, CRISP-DM menyediakan enam fasa iteratif yang bersifat non-linear: (1) Pemahaman Masalah (*Business/Research Understanding*), yaitu pendefinisian tujuan analisis dan kriteria keberhasilan model; (2) Pemahaman Data (*Data Understanding*), yaitu eksplorasi awal struktur dan kualitas korpus data; (3) Persiapan Data (*Data Preparation*), yaitu transformasi dan pembersihan atribut data mentah menjadi representasi fitur siap latih; (4) Pemodelan (*Modeling*), yaitu seleksi dan pelatihan algoritma pembelajaran mesin; (5) Evaluasi (*Evaluation*), yaitu pengujian akurasi serta generalisasi model berdasarkan metrik kuantitatif; dan (6) Penerapan (*Deployment*), yaitu pengintegrasian model atau artefak ke dalam lingkungan produksi (Shearer, 2000). Kerangka teoritis CRISP-DM ini menjadi landasan rekayasa pemodelan yang menggantikan model konvensional linear.

2.5.3 Privacy by Design dan Keamanan Data

Dalam penelitian yang memproses data kualitatif sensitif, perlindungan privasi informasi subjek merupakan persyaratan etis dan legal yang utama. Kerangka kerja *Privacy by Design* (Cavoukian, 2011) menempatkan perlindungan privasi sebagai standar fundamental yang terintegrasi di dalam arsitektur sistem sejak tahap awal perancangan, bukan sekadar fitur pertahanan yang ditambahkan di akhir siklus pengembangan (Cavoukian, 2011). Prinsip ini sejalan dengan mandat kepatuhan hukum pada Undang-Undang Pelindungan Data Pribadi (UU PDP) di Indonesia (Republik Indonesia, 2022). Dalam teori rekayasa perangkat lunak aman, penerapan *Privacy by Design* menuntut adanya isolasi pemrosesan di dalam infrastruktur mandiri (*on-premise*) serta gerbang verifikasi persetujuan (*consent-based privacy gate*) yang mencegah eksekusi data yang belum terautentikasi keamanannya.

2.6 Penelitian Terkait

Kajian literatur terhadap penelitian terkait dilakukan untuk memetakan posisi penelitian ini di antara karya-karya ilmiah sebelumnya, mengidentifikasi celah metodologis, serta mempertegas keunggulan komparatif dari rancangan komponen *ML Service* yang dibangun. Dalam perkembangan disiplin ilmu *Automated Machine Learning* (AutoML), penelitian seminal oleh Feurer et al. (2015) meletakkan fondasi penting melalui otomatisasi pemilihan algoritma

dan optimasi hiperparameter pada data tabular (Feurer et al., 2015). Karya tersebut diperluas oleh kajian survei dan studi patok uji (*benchmark*) oleh Zöller dan Huber (2021) yang mengevaluasi performa berbagai kerangka kerja *AutoML* bersumber terbuka (Zoller & Huber, 2021). Namun, kedua karya ilmiah tersebut masih memiliki celah arsitektural: otomatisasi umumnya berfokus pada pemrosesan modalitas tabular tunggal di dalam lingkungan komputasi monolitik, serta tidak merancang arsitektur layanan mandiri (*decoupled microservice*) maupun mekanisme perlindungan privasi data penelitian.

Di sisi lain, perkembangan teknologi *Large Language Model* (LLM) telah mendorong otomatisasi analisis teks dalam penelitian kualitatif. Studi pemetaan sistematis oleh Barros et al. (2025) memetakan penerapan LLM untuk tugas pengodean tematik dan sintesis narasi wawancara kualitatif (Barros et al., 2025). Sementara itu, Albert dan Voutama (2025) menerapkan inferensi LLM secara lokal menggunakan platform Ollama guna mendukung privasi data pada sistem tanya jawab berbasis dokumen (Albert & Voutama, 2025). Kendati memberikan kontribusi penting dalam aspek perlindungan data sensitif tanpa ketergantungan biaya token komputasi awan, penelitian-penelitian tersebut berfokus pada tugas pemrosesan bahasa alami tunggal dan tidak mengintegrasikan analisis LLM kualitatif ke dalam platform *AutoML* terpadu yang juga menangani pemodelan *supervised learning* multi-modalitas.

Penelitian ini menutup celah literatur di atas melalui rancangan komponen *ML Service* pada *AutoML Platform v2* yang memadukan otomatisasi alur kerja pemelajaran mesin klasik dan analisis semantik LLM lokal. Keunggulan komparatif penelitian ini terletak pada dukungan *pipeline preprocessing* untuk tiga modalitas data sekaligus (tabular, citra HOG, dan teks TF-IDF), penerapan kontrak keluaran artefak terstandardisasi (*X.pkl*, *y.pkl*, dan *meta.json*), integrasi inferensi Ollama berlandaskan prinsip *Privacy by Design* dan efisiensi biaya, serta pengujian keterulangan bereksperimen (*reproducibility*). Perbandingan sistematis antara penelitian ini dan penelitian-penelitian terkait dirangkum pada Tabel 2.1.

Tabel 2.1: Ringkasan Penelitian Terkait

Penelitian	Fokus Penelitian	Celah & Keunggulan Komparatif Penelitian Ini
Feurer et al. (2015)	AutoML dan optimasi hiperparameter pipeline tabular	Hanya mendukung data tabular tunggal secara monolitik; tidak merancang arsitektur microservice atau integrasi LLM.
Zöller & Huber (2021)	Survei komparatif dan patok uji framework AutoML	Berfokus pada evaluasi pustaka siap pakai; tidak memiliki kontrak artefak multi-modalitas atau gerbang privasi data.
Barros et al. (2025)	Studi pemetaan sistematis penggunaan LLM pada riset kualitatif	Merupakan kajian literatur pemetaan tanpa arsitektur runtime lokal berskala produksi ataupun integrasi AutoML.
Albert & Voutama (2025)	Inferensi LLM lokal Ollama untuk sistem tanya jawab dokumen PDF	Berfokus pada pemrosesan dokumen tunggal (RAG); tidak memiliki alur preprocessing citra/tabular atau pelatihan model.
Penelitian Ini (AutoML Platform v2)	ML Service multi-modalitas, pelatihan model modular, dan analisis kualitatif LLM lokal	Menyediakan solusi terpadu microservice AutoML klasik dengan inferensi LLM lokal yang aman privasi, hemat biaya, dan terstandardisasi.

Bab 3. METODE PENELITIAN

3.1 Pendekatan Design Science Research

Penelitian ini menerapkan pendekatan *Design Science Research* (DSR) berlandaskan kerangka kerja metodologis Peffers et al. (2007) (Peffers et al., 2007). Dalam disiplin ilmu teknik informatika dan rekayasa perangkat lunak sains data, pendekatan DSR dipilih karena penelitian tidak sekadar bertujuan melakukan pengujian hipotesis statistik pada sistem konvensional, melainkan berfokus pada perancangan, pembangunan, dan evaluasi artefak keilmuan inovatif guna memecahkan permasalahan empiris yang relevan. Artefak utama yang dihasilkan dalam penelitian ini berupa rancangan arsitektur terdesentralisasi komponen *ML Service* beserta integrasi alur pemodelannya di dalam platform *AutoML Platform v2*.

Metodologi DSR memberikan landasan epistemologis yang terstruktur dalam mengorkestrasi tahapan investigasi ilmiah melalui enam fasa formal yang saling iteratif. Penerapan keenam tahapan DSR pada pengembangan komponen *ML Service* diuraikan sebagai berikut:

1. Identifikasi Masalah dan Motivasi (*Problem Identification and Motivation*): Tahap ini menganalisis hambatan teknis yang dihadapi peneliti non-teknis dalam membangun alur kerja pemodelan *machine learning* multi-modalitas (tabular, citra, dan teks). Selain itu, diidentifikasi keterbatasan arsitektur monolitik konvensional yang rentan mengalami pemblokiran *thread* akibat beban komputasi intensif (*CPU-bound*, *memory-bound*, dan *GPU-bound*), serta urgensi perlindungan privasi data kualitatif sensitif.
2. Penetapan Tujuan Solusi (*Define Objectives for a Solution*): Berdasarkan identifikasi masalah, dirumuskan spesifikasi tujuan solusi berupa perancangan arsitektur layanan mandiri (*decoupled microservice*), kontrak

artefak keluaran yang terstandarisasi (X.pkl, y.pkl, meta.json, dan laporan status *completed/failed*), penerapan gerbang keamanan berlandaskan *Privacy by Design*, serta efisiensi biaya operasional melalui inferensi *Large Language Model* (LLM) lokal tanpa biaya token komputasi awan.

3. Perancangan dan Pengembangan (*Design and Development*): Tahap ini merancang dan mewujudkan kode sumber operasional komponen *ML Service*. Pengembangan mencakup pembangunan modul *pipeline preprocessing* untuk data CSV tabular, citra digital HOG, dan teks TF-IDF, perancangan modul pengelolaan eksperimen dan pelatihan model *supervised learning* Scikit-learn berbasis pola desain pabrik (*factory pattern*), serialisasi artefak, serta integrasi metode *few-shot prompting* untuk analisis kualitatif LLM Ollama.
4. Demonstrasi (*Demonstration*): Artefak komponen *ML Service* diterapkan untuk mengeksekusi serangkaian skenario pemodelan nyata. Demonstrasi meliputi pengujian alur klasifikasi dan regresi pada data tabular, klasifikasi kumpulan citra berkelas, analisis sentimen teks, serta eksekusi pengodean tematik dan sintesis narasi kualitatif pada dataset transkrip wawancara (llm_raw dan llm_new).
5. Evaluasi (*Evaluation*): Kinerja artefak divalidasi melalui pengujian sistematis terhadap fungsionalitas, keandalan jalur sukses dan gagal, serta jaminan keterulangan bereksperimen (*reproducibility*). Evaluasi kuantitatif model dilakukan dengan menghitung metrik klasifikasi (*Accuracy, Precision, Recall, F1-Score*), metrik regresi (MSE, RMSE, MAE, R-squared), serta skor kesamaan leksikal ROUGE.
6. Komunikasi (*Communication*): Seluruh rancangan arsitektur, spesifikasi antarmuka pemrograman aplikasi (API), matriks pengujian, dan hasil evaluasi empiris didokumentasikan ke dalam karya ilmiah skripsi ini serta disiapkan untuk diseminasi pada publikasi ilmiah terkait.

Tabel 3.1: Implementasi Tahapan Design Science Research (DSR) pada Proyek AutoML Platform v2

Tahap DSR	Aktivitas Penelitian & Rekayasa	Artefak & Keluaran Keilmuan
Identifikasi Masalah	Menganalisis hambatan peneliti non-teknis, beban komputasi monolitik, dan risiko privasi data kualitatif.	Pernyataan masalah dan justifikasi arsitektur terdesentralisasi.
Tujuan Solusi	Merumuskan spesifikasi layanan microservice mandiri, kontrak artefak standar, gerbang privasi, dan efisiensi biaya.	Spesifikasi kontrak antarmuka API dan target kinerja model.
Desain & Pengembangan	Membangun modul preprocessing tabular, citra HOG, teks TF-IDF, training modular Scikit-learn, dan inferensi LLM Ollama.	Kode sumber operasional komponen ML Service.
Demonstrasi	Mengeksekusi skenario eksperimen multi-modalitas dan pengodean tematik transkrip wawancara kualitatif.	Log eksekusi, artefak data (X.pkl, y.pkl), dan model terlatih.
Evaluasi	Memvalidasi akurasi, presisi, recall, F1, kesalahan regresi, skor ROUGE, dan jaminan keterulangan eksperimen.	Matriks hasil evaluasi dan berkas metadata meta.json.
Komunikasi	Mendokumentasikan seluruh arsitektur, metode, dan bukti empiris ke dalam naskah karya ilmiah skripsi.	Dokumen skripsi, dokumentasi API, dan lampiran teknis.

3.2 Metodologi Rekayasa Pemodelan CRISP-DM

Dalam pengembangan sistem perangkat lunak konvensional, model siklus hidup linear seperti *Waterfall* atau *Systems Development Life Cycle* (SDLC) sering digunakan karena persyaratan sistem bersifat statis dan deterministik. Namun, dalam ranah rekayasa sains data dan pembelajaran mesin (*Machine Learning Engineering*), penerapan model linear konvensional memiliki keterbatasan mendasar. Pembangunan model *machine learning* bersifat empiris, eksploratif, iteratif, dan sangat bergantung pada karakteristik distribusi data (*data-driven and exploratory*). Alur kerja tidak dapat dikunci pada fase desain statis di awal, melainkan membutuhkan pengulangan terus-menerus antara eksplorasi fitur, pelatihan estimator, evaluasi performa, hingga penyempurnaan kualitas data.

Berdasarkan pertimbangan ilmiah tersebut, proses rekayasa pemodelan pada komponen *ML Service* mengadopsi metodologi *Cross-Industry Standard Process for Data Mining* (CRISP-DM) (Shearer, 2000). CRISP-DM menyedia-

kan enam fasa iteratif yang diselaraskan secara operasional dengan alur kerja pada platform *AutoML Platform v2*:

1. Pemahaman Tujuan Penelitian (*Business/Research Understanding*): Tahap awal yang menerjemahkan kebutuhan analisis peneliti non-teknis menjadi spesifikasi kontrak antarmuka API komponen *ML Service*, serta menetapkan target kinerja model (seperti akurasi klasifikasi atau ambang batas skor F-measure ROUGE).
2. Pemahaman Data (*Data Understanding*): Tahap pemindaian awal terhadap karakteristik struktur data masukan yang diterima dari server *backend*, meliputi verifikasi skema kolom pada berkas CSV tabular, inspeksi struktur subfolder kelas pada direktori ZIP citra digital, dan pemeriksaan kelengkapan naskah transkrip TXT.
3. Persiapan Data (*Data Preparation*): Tahap perancangan dan implementasi *pipeline preprocessing* multi-modalitas. Data tabular diproses melalui imputasi, penskalaan *StandardScaler* atau *MinMaxScaler*, dan pengodean kategorikal. Data citra diproses melalui standardisasi resolusi dan ekstraksi fitur *Histogram of Oriented Gradients* (HOG) (Dalal & Triggs, 2005). Data teks diproses melalui pembersihan karakter dan vektorisasi *Term Frequency-Inverse Document Frequency* (TF-IDF) (Salton & Buckley, 1988). Seluruh transformasi digabungkan menggunakan *ColumnTransformer* dan diserialisasi ke dalam berkas *preprocessor.pkl*.
4. Pemodelan (*Modeling*): Tahap pelatihan estimator klasifikasi dan regresi Scikit-learn (*Decision Tree*, *Random Forest*, *SVM/SVR*, *k-NN*, dan *Logistic Regression*) berlandaskan pola desain pabrik (*factory pattern*) (Pedregosa et al., 2011). Selain pemodelan klasik, tahap ini juga menjalankan inferensi semantik lokal menggunakan platform Ollama berbasis *few-shot prompting* untuk data kualitatif.
5. Evaluasi (*Evaluation*): Tahap validasi kinerja estimator atas data pengujian dengan proporsi pembagian 80 persen latih dan 20 persen uji menggunakan parameter acak statis guna menjamin keterulangan bereksperimen (*reproducibility*). Sistem menghitung metrik klasifikasi (*Accuracy*, *Precision*, *Recall*, *F1-Score*), kesalahan regresi (MSE, RMSE,

MAE, R-squared), serta kemiripan leksikal ROUGE, lalu menyandikannya ke dalam berkas `meta.json`.

- 6. Penerapan dan Integrasi Layanan (*Deployment/Service Integration*): Tahap pengemasan seluruh artefak hasil pemodelan (`model.pkl`, `preprocessor.pkl`, dan `meta.json`) ke dalam struktur direktori modular agar siap diakses, dipantau, dan diorkestrasi secara asinkron oleh server *backend* utama melalui **endpoint** API.

Tabel 3.2: Pemetaan Fasa Iteratif CRISP-DM terhadap Modul Operasional Komponen ML Service

Fasa CRISP-DM	Implementasi Modul ML Service	Artefak & Kontrak Keluaran
Business Understanding	Perumusan skema antarmuka REST API dan parameter konfigurasi eksperimen.	Skema JSON payload dan kontrak status completed/failed.
Data Understanding	Pemindaian skema kolom CSV, validasi subfolder kelas citra ZIP, dan inspeksi transkrip TXT.	Laporan validasi skema data dan pendeteksian nilai hilang.
Data Preparation	Eksekusi imputasi, scaling, ekstraksi HOG, dan vektorisasi TF-IDF berbasis ColumnTransformer.	Matriks fitur X.pkl, label y.pkl, dan berkas preprocessor.pkl.
Modeling	Pelatihan estimator Scikit-learn via factory pattern dan inferensi LLM lokal Ollama.	Objek model terlatih model.pkl dan teks keluaran analisis semantik.
Evaluation	Penghitungan Accuracy, Precision, Recall, F1, MSE, RMSE, MAE, R2, dan skor ROUGE.	Struktur JSON metrik performa di dalam berkas meta.json.
Deployment / Integration	Pengemasan artefak pemodelan ke direktori modular yang dapat diakses oleh server backend.	Paket artefak eksperimen modular siap pakai untuk layanan inferensi.

3.3 Batas Kontribusi Tim

Data Engineer menangani upload dataset dan consent. Data Scientist menangani konfigurasi preprocessing dan analisis. ML Engineer menangani pipeline ML, training, packaging, dan integrasi analisis LLM. Backend Engineer mengelola API, basis data, autentikasi, dan orkestrasi, sedangkan Frontend Engineer membangun antarmuka.

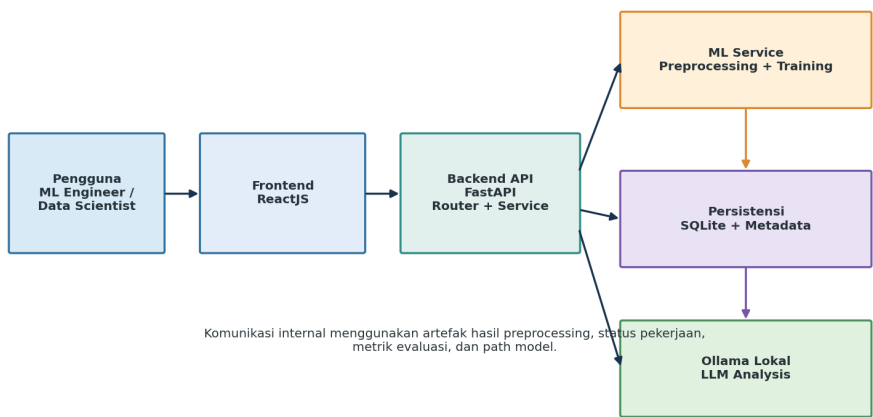
Tabel 3.3: Pembagian Komponen

Komponen	Kontribusi ML Engineer	Integrasi
Preprocessing	Tabular, gambar, teks, metadata, artefak.	run_preprocessing_job.
Training	Factory estimator, split, fit, evaluasi, packaging.	run_training_job.
LLM	Prompt, parsing, ROUGE, waktu, panjang.	Endpoint analysis dan Ollama.
Privacy gate	Menghormati dataset locked.	Validasi sebelum pipeline.

3.4 Arsitektur Sistem

Frontend mengirim permintaan ke endpoint. Router memvalidasi autentikasi dan role, service membuat record pekerjaan, dan background task memanggil ML Service. Hasil disimpan sebagai file dan metadata pada basis data. LLM lokal diakses melalui endpoint Ollama.

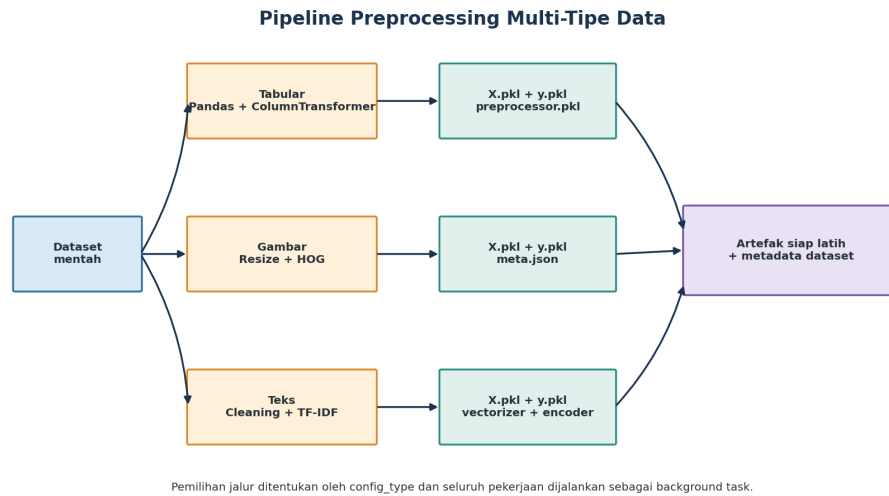
Arsitektur Komponen ML Engineer pada AutoML Platform v2



Gambar 3.1: Arsitektur komponen ML Engineer pada AutoML Platform v2

3.5 Rancangan Pipeline Preprocessing

Pipeline menerima dataset_path, config, dan output_path. Pemilihan fungsi berdasarkan config_type. Setiap fungsi mengembalikan status dan meta. Jika gagal, error diteruskan ke record pekerjaan.



Gambar 3.2: Pipeline preprocessing tiga tipe data

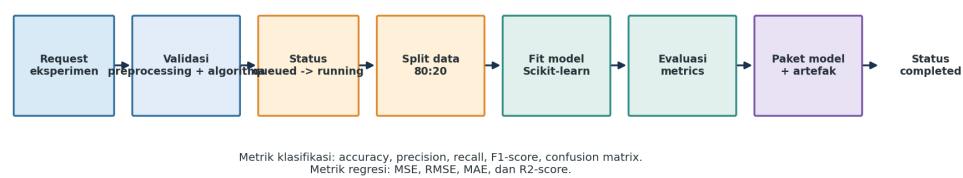
Tabel 3.4: Konfigurasi dan Artefak

Tipe	Konfigurasi	Artefak
Tabular	target, fitur, missing, scaling, encoding, duplikat	X.pkl, y.pkl, preprocessor.pkl, target_encoder.pkl, meta.json
Gambar	image_size, grayscale, label_groups	X.pkl, y.pkl, meta.json
Teks	labels, edited_texts, stopword, language, max_features	X.pkl, y.pkl, vectorizer.pkl, label_encoder.pkl, meta.json

3.6 Rancangan Eksperimen dan Training

Pembuatan eksperimen memeriksa preprocessing completed dan algoritma yang diizinkan. Dataset gambar dan teks otomatis menjadi classification. Training memuat X dan y, membagi data 80:20, membuat estimator, fit, prediksi, menghitung metrik, dan menggabungkan model dengan artefak transformasi.

Alur Eksperimen dan Training Model



Gambar 3.3: Alur eksperimen dan training

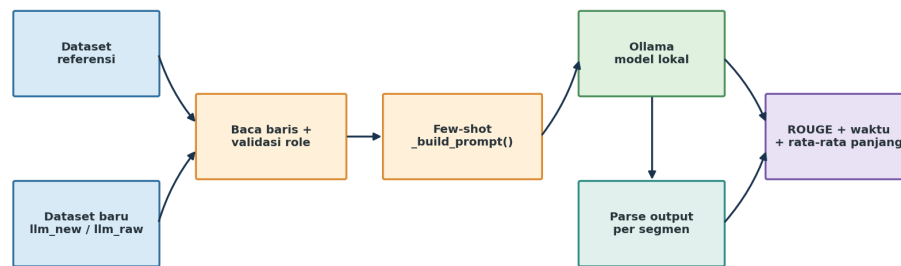
Tabel 3.5: Matriks Algoritma

Data	Task	Algoritma
Tabular	Classification	Decision Tree, Random Forest, SVM, KNN, Logistic Regression
Tabular	Regression	Linear Regression, Decision Tree, Random Forest, SVR
Image	Classification	SVM, Random Forest, KNN
Text	Classification	Naive Bayes, Logistic Regression, SVM, Random Forest

3.7 Rancangan Analisis LLM

Dataset referensi wajib memiliki role llm_reference. Dataset baru dapat llm_new atau llm_raw. Prompt memuat tiga contoh referensi dan format output. Output Ollama dipecah berdasarkan penanda SEGMEN, dipetakan ke input, lalu disimpan sebagai JSON bersama ROUGE, waktu, dan panjang rata-rata.

Alur Analisis LLM Lokal dengan Ollama



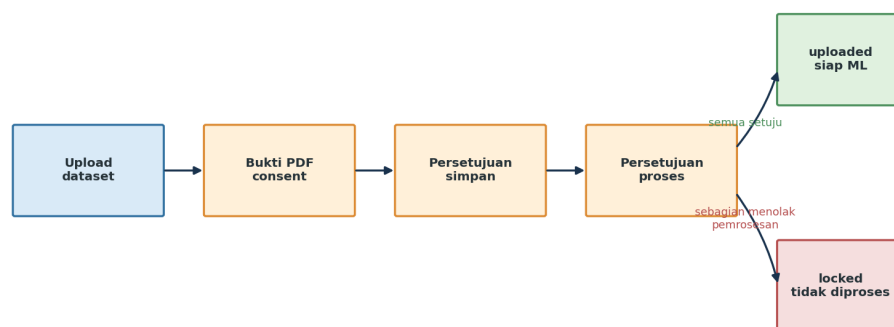
Hasil disimpan pada tabel llm_analyses dan dapat dipantau melalui endpoint status.

Gambar 3.4: Alur analisis LLM lokal dan evaluasi ROUGE

3.8 Rancangan Privacy Gate

Dataset locked ditolak sebelum preprocessing atau analisis. Status locked berasal dari alur consent ketika terdapat subjek yang menolak pemrosesan; persetujuan penyimpanan yang tidak lengkap menyebabkan dataset dihapus.

Perlindungan Dataset sebelum Diproses



NIK dienkripsi dengan Fernet sebelum disimpan; data tidak diproses ketika status dataset locked.

Gambar 3.5: Gate persetujuan sebelum dataset diproses

3.9 Pemilihan Teknologi dan Pengujian

Tabel 3.6: Teknologi

Teknologi	Peran
Python	Bahasa ML Service.
Pandas dan NumPy	Manipulasi data dan array.
Scikit-learn	Preprocessing, estimator, dan metrics.
Pillow dan scikit-image	Pembacaan gambar dan HOG.
NLTK	Stopword.
Joblib	Serialisasi artefak.
FastAPI BackgroundTasks	Pekerjaan asinkron.
Ollama dan HTTPX	LLM lokal dan komunikasi HTTP.

Pengujian menggunakan white box berbasis skenario. Selain response API, diperiksa pula status record, keberadaan file, metadata, metrik, dan pesan error.

Bab 4. HASIL DAN PEMBAHASAN

4.1 Lingkungan Implementasi

Implementasi berada pada repositori AutoML Platform v2. Backend memakai FastAPI dan SQLAlchemy, sementara fungsi ML berada pada direktori `ml_service`. Dependensi utama adalah pandas, NumPy, Scikit-learn, Pillow, scikit-image, NLTK, dan joblib. Runtime LLM diakses melalui Ollama lokal.

Tabel 4.1: Berkas Fokus

Berkas	Fungsi
<code>preprocessing/tabular.py</code>	ColumnTransformer dan artefak tabular.
<code>preprocessing/image.py</code>	Resize dan HOG.
<code>preprocessing/text.py</code>	Cleaning, TF-IDF, dan label.
<code>training/pipeline.py</code>	Factory, training, evaluasi, packaging.
<code>services/preprocessing_service.py</code>	Orkestrasi preprocessing.
<code>services/experiment_service.py</code>	Orkestrasi training.
<code>routers/analysis.py</code>	Prompt, Ollama, parsing, ROUGE.

4.2 Implementasi Preprocessing Tabular

Fungsi `run_tabular_preprocessing` membaca data, memvalidasi konfigurasi, menghapus duplikat, memisahkan fitur-target, menentukan tipe kolom, membangun pipeline numerik dan kategorikal, fit-transform, lalu menyimpan metadata.

```
df = pd.read_csv(dataset_path, sep=None, engine="python")
target_column = config.get("target_column")
```

```

feature_columns = config.get("feature_columns", [])
if target_column not in df.columns:
    return {"status": "failed", "error": "target_tidak_ditemukan"}
preprocessor = ColumnTransformer(transformers)
X_processed = preprocessor.fit_transform(X)
joblib.dump(X_processed, os.path.join(output_path, "X.pkl"))
joblib.dump(preprocessor, os.path.join(output_path, "preprocessor.pkl"))

```

Catatan teknis: pada implementasi saat ini preprocessor di-fit sebelum split data. Untuk evaluasi yang bebas data leakage, transformasi perlu dibungkus bersama estimator dan hanya di-fit pada data training.

4.3 Implementasi Preprocessing Gambar

Fungsi `run_image_preprocessing` membangun mapping folder ke label, memindai ekstensi valid, mengonversi gambar, melakukan resize, dan mengekstraksi HOG. Jika tidak ada gambar valid, status failed dikembalikan.

```

img = Image.open(img_path)
img = img.convert("L" if grayscale else "RGB")
img = img.resize((image_size, image_size))
features = hog(
    np.array(img), orientations=8,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    channel_axis=None if grayscale else -1,
)

```

4.4 Implementasi Preprocessing Teks

Fungsi `run_text_preprocessing` menerapkan `edited_texts`, cleaning, stopword, TF-IDF, dan `LabelEncoder`. Minimal dua kelas diperlukan. Matriks TF-IDF diubah menjadi dense array agar kontrak training seragam, tetapi hal ini dapat meningkatkan memori pada korpus besar.

```

cleaned = _clean_text(text)
vectorizer = TfidfVectorizer(max_features=max_features, min_df=1)
X = vectorizer.fit_transform(cleaned_texts).toarray()
le = LabelEncoder()
y = le.fit_transform(y_labels_raw)

```

4.5 Implementasi Training dan Evaluasi

Fungsi `run_training` memuat `X` dan `y` serta `meta.json`, melakukan split 80:20, membuat model melalui `factory`, melakukan fit dan prediksi, memilih evaluator klasifikasi atau regresi, lalu menyimpan paket model.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = _get_model(data_type, task_type, algorithm, hyperparameters)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
metrics = _evaluate_classification(y_test, y_pred)
```

4.6 Integrasi Background Task

Endpoint membuat record pending atau queued, mendaftarkan fungsi pekerjaan, lalu segera mengembalikan ID. Background task membuka sesi basis data, mengubah status running, memanggil ML Service, menyimpan hasil, dan mengubah status completed atau failed.

Tabel 4.2: Transisi Status

Pekerjaan	Sukses	Gagal
Preprocessing	pending ke running ke completed	pending atau running ke failed
Eksperimen	queued ke running ke completed	queued atau running ke failed
Analisis LLM	pending ke running ke completed	pending atau running ke failed

4.7 Implementasi Analisis LLM

Fungsi `_read_dataset_rows` membaca dataset sesuai tipe dan role. `_build_prompt` memasukkan contoh referensi dan format keluaran. `_run_analysis` melakukan request asynchronous ke Ollama, mengukur waktu, mem-parse keluaran, menghitung ROUGE, lalu menyimpan hasil.

```
prompt = "Kamu adalah psikolog klinis profesional..."
for i, ex in enumerate(examples, 1):
    prompt += f"CONTOH_{i}:"
    prompt += f"Verbatim : {ex.get('verbatim', '')}"
```

```

    prompt += f"Analisis_{ex.get('analisis', '')}"
if dataset_role=="llm_raw":
    prompt+="Buatkan CODING dan ANALISIS..."
else:
    prompt+="Buatkan ANALISIS..."

```

```

scores = scorer.score(ref_combined, hyp_combined)
return {
    "rouge1": round(scores["rouge1"].fmeasure, 4),
    "rouge2": round(scores["rouge2"].fmeasure, 4),
    "rougeL": round(scores["rougeL"].fmeasure, 4),
}

```

4.8 Pengujian Preprocessing

Tabel 4.3: Pengujian Preprocessing

Fungsi	Skenario	Harapan	Hasil
Tabular	Target dan fitur valid	X, y, preprocessor, metadata tersedia	Sesuai
Tabular	Target tidak tersedia	Status failed	Sesuai
Gambar	Folder kelas berisi gambar	Fitur HOG dan label tersimpan	Sesuai
Gambar	Tidak ada gambar valid	Status failed	Sesuai
Teks	Minimal dua kelas	TF-IDF dan encoder tersimpan	Sesuai
Teks	Label kosong atau satu kelas	Status failed	Sesuai

4.9 Pengujian Eksperimen dan Training

Tabel 4.4: Pengujian Training

Fungsi	Skenario	Harapan	Hasil
create_experiment	Preprocessing completed, algoritma valid	Record queued dan task dibuat	Sesuai
create_experiment	Preprocessing running	HTTP 400	Sesuai
create_experiment	Algoritma tidak valid	HTTP 400	Sesuai
run_training	X dan y tersedia	Model dan metrics tersimpan	Sesuai
download_model	Completed dan file ada	File.pkl dikirim	Sesuai

4.10 Pengujian Analisis LLM

Tabel 4.5: Pengujian LLM

Fungsi	Skenario	Harapan	Hasil
generate_analysis	Role dataset benar dan model ada	Record pending dibuat	Sesuai
generate_analysis	Referensi role general	HTTP 400	Sesuai
generate_analysis	Dataset baru locked	HTTP 403	Sesuai
_build_prompt	llm_raw	Meminta coding dan analisis	Sesuai
_build_prompt	llm_new	Meminta analisis	Sesuai
_calculate_rouge	Referensi dan hasil ada	ROUGE dalam rentang 0 sampai 1	Sesuai

4.11 Pembahasan dan Keterbatasan

Tiga tipe data dapat diperlakukan melalui kontrak seragam berupa status, meta, dan path artefak meskipun transformasi internal berbeda. Packaging model bersama artefak preprocessing mengurangi risiko ketidaksesuaian transformasi ketika model digunakan kembali.

Background task menjaga endpoint tetap responsif, tetapi belum menggantikan message queue untuk beban produksi besar. Jika proses aplikasi berhenti, pekerjaan dapat terhenti dan retry belum tersedia.

Analisis LLM lokal tidak bergantung pada API cloud, tetapi waktu generate dipengaruhi model, hardware, panjang prompt, dan jumlah segmen. ROUGE memberi indikator tekstual dan tidak otomatis menyatakan analisis kualitatif benar.

- Repositori tidak menyertakan dataset benchmark final sehingga skor accuracy, F1, RMSE, dan ROUGE harus diisi setelah eksperimen.
- Pembagian data dilakukan sekali dan belum menggunakan cross-validation.
- Fit preprocessing sebelum split berpotensi data leakage untuk evaluasi formal.
- Belum terdapat worker queue eksternal, retry, cancellation, atau isolasi resource.

- Belum ada penilaian ahli untuk validitas semantik keluaran LLM.

Bab 5. PENUTUP

5.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian, kesimpulan penelitian adalah:

1. ML Service berhasil dirancang untuk tabular, gambar, dan teks dengan pipeline spesifik serta artefak X, y, metadata, dan transformasi.
2. Modul eksperimen dan training berhasil memvalidasi konfigurasi, memilih estimator, melakukan split dan training, menghitung metrik, serta mengemas model.
3. Analisis LLM lokal berhasil diintegrasikan dengan Ollama, few-shot prompting, parsing, pengukuran waktu, rata-rata panjang, dan ROUGE.
4. Skenario pengujian memverifikasi jalur valid dan error utama. Dataset locked ditolak sebelum preprocessing atau analisis.
5. Pemisahan ML Service dari API membantu menjaga keterpisahan tanggung jawab dan memudahkan penambahan algoritma.

5.2 Saran

Berdasarkan kesimpulan dan keterbatasan metodologis yang ditemukan selama penelitian, berikut adalah saran penelitian lanjutan yang dapat menjadi acuan bagi peneliti berikutnya:

1. Disarankan pada penelitian lanjutan agar penerapan *pipeline preprocessing* (terutama transformasi *scaling* pada data tabular dan *vectorization* pada data teks) dipisahkan secara ketat sehingga proses *fitting*

hanya dilakukan pada partisi data latih (*training set*) dalam skema *k-fold cross-validation* guna menghindari *data leakage*.

2. Penelitian berikutnya dapat mengeksplorasi metode optimasi hiperparameter adaptif (seperti *Bayesian Optimization*) serta menerapkan pengujian berbasis *seed reproducibility* yang terstandarisasi untuk menganalisis variansi performa model antar-eksperimen.
3. Evaluasi terhadap analisis *Large Language Model* (LLM) lokal disarankan tidak hanya mengacu pada metrik *n-gram similarity* seperti ROUGE, melainkan dikombinasikan dengan metode *human-in-the-loop evaluation* oleh pakar (*expert judgment*) guna mengukur akurasi semantik dan meminimalkan halusinasi model.
4. Penelitian lanjutan dapat memperluas cakupan eksperimen *AutoML* ke arsitektur *deep learning* (seperti PyTorch atau TensorFlow) untuk membandingkan efisiensi komputasi dan performa prediksi terhadap model klasik Scikit-learn, khususnya pada tipe data non-konvensional seperti deret waktu (*time-series*) dan audio.

DAFTAR PUSTAKA

- Albert, G. D., & Voutama, A. (2025). Pengembangan chatbot berbasis pdf menggunakan local retrieval-augmented generation (rag) dan ollama. *Jurnal Informatika dan Teknik Elektro Terapan*, 13(2). doi: 10.23960/jitet.v13i2.6361
- Barros, C. F., Azevedo, B. B., Graciano Neto, V. V., Kassab, M., Kalinowski, M., Nascimento, H. A. D., & Bandeira, M. C. G. S. P. (2025). *Large language model for qualitative research: A systematic mapping study*. arXiv preprint arXiv:2411.14473. doi: 10.48550/arXiv.2411.14473
- Bergstra, J., Bardenet, R., Bengio, Y., & Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in neural information processing systems* (Vol. 24, pp. 2546–2554).
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. doi: 10.1023/A:1010933404324
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... others (2020). Language models are few-shot learners. In *Advances in neural information processing systems* (Vol. 33, pp. 1877–1901).
- Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., ... Varoquaux, G. (2013). Api design for machine learning software: Experiences from the scikit-learn project. In *Ecml pkdd workshop: Languages for data mining and machine learning* (pp. 108–122).
- Cavoukian, A. (2011). Privacy by design: The 7 foundational principles. *Information and Privacy Commissioner of Ontario*. Retrieved from <https://www.ipc.on.ca/wp-content/uploads/resources/7foundationalprinciples.pdf>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. doi: 10.1007/BF00994018
- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE*

- Transactions on Information Theory*, 13(1), 21–27. doi: 10.1109/TIT.1967.1053964
- Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (Vol. 1, pp. 886–893). doi: 10.1109/CVPR.2005.177
- Feurer, M., Klein, A., Eggenberger, K., Springenberg, J., Blum, M., & Hutter, F. (2015). Efficient and robust automated machine learning. In *Advances in neural information processing systems* (Vol. 28, pp. 2962–2970).
- Giraldo, L., & Laso, S. (2025). Democratizing machine learning: A practical comparison of low-code and no-code platforms. *Machine Learning and Knowledge Extraction*, 7(4), 141. doi: 10.3390/make7040141
- Hutter, F., Kotthoff, L., & Vanschoren, J. (Eds.). (2019). *Automated machine learning: Methods, systems, challenges*. Springer. doi: 10.1007/978-3-030-05318-5
- kiki-kisaki. (2026). *Automl platform v2*. Repositori perangkat lunak. Retrieved from <https://github.com/kiki-kisaki/automl-platform-v2> (Diakses 9 Agustus 2026)
- Kowsari, K., Jafari Meimandi, K., Heidarysafa, M., Mendu, S., Barnes, M., & Brown, D. E. (2019). Text classification algorithms: A survey. *Information*, 10(4), 150. doi: 10.3390/info10040150
- Lin, C.-Y. (2004). Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out* (pp. 74–81). Association for Computational Linguistics.
- Ollama. (2026). *Ollama: Get up and running with large language models*. Dokumentasi daring. Retrieved from <https://ollama.com/> (Diakses 9 Agustus 2026)
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830. Retrieved from <https://jmlr.org/papers/v12/pedregosa11a.html>
- Peffer, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. doi: 10.2753/MIS0742-1222240302
- Python Software Foundation. (2026). *Python 3 documentation*. Dokumen-

- tasi daring. Retrieved from <https://docs.python.org/3/> (Diakses 9 Agustus 2026)
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106. doi: 10.1007/BF00116251
- Republik Indonesia. (2022). *Undang-undang republik indonesia nomor 27 tahun 2022 tentang perlindungan data pribadi*. Lembaran Negara Republik Indonesia Tahun 2022 Nomor 196. Retrieved from <https://peraturan.bpk.go.id/Details/229798/uu-no-27-tahun-2022>
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing and Management*, 24(5), 513–523. doi: 10.1016/0306-4573(88)90021-0
- Shearer, C. (2000). The crisp-dm model: the new blueprint for data mining. *Journal of Data Warehousing*, 5(4), 13–22.
- Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval augmentation reduces hallucination in conversation. *arXiv preprint arXiv:2104.07567*. doi: 10.48550/arXiv.2104.07567
- Zoller, M.-A., & Huber, M. F. (2021). Benchmark and survey of automated machine learning frameworks. *Journal of Artificial Intelligence Research*, 70, 409–472. doi: 10.1613/jair.1.11854

LAMPIRAN

Lampiran 1 Listing Program ML Service

Potongan kode berikut diringkas dari repositori AutoML Platform v2.

```
def _evaluate_classification(y_true, y_pred):  
    return {  
        "accuracy": round(accuracy_score(y_true, y_pred), 4),  
        "precision": round(precision_score(  
            y_true, y_pred, average="weighted", zero_division=0), 4),  
        "recall": round(recall_score(  
            y_true, y_pred, average="weighted", zero_division=0), 4),  
        "f1_score": round(f1_score(  
            y_true, y_pred, average="weighted", zero_division=0), 4),  
        "confusion_matrix": confusion_matrix(y_true, y_pred).tolist(),  
    }
```

```
{  
    "status": "completed",  
    "meta": {  
        "n_samples": 100,  
        "n_features": 12,  
        "classes": ["negatif", "positif"]  
    }  
}
```

Lampiran 2 Contoh Konfigurasi Eksperimen

```
{  
    "config_type": "tabular",  
    "target_column": "label",  
    "feature_columns": ["usia", "skor", "kategori"],
```

```
"missing_strategy": "median",  
"scaling": "standard",  
"encoding": "onehot",  
"duplicate_strategy": "drop"  
}
```

```
{  
  "preprocessing_id": 1,  
  "name": "RF tabular baseline",  
  "task_type": "classification",  
  "algorithm": "random_forest",  
  "hyperparameters": {"n_estimators": 100, "random_state": 42}  
}
```

Lampiran 3 Checklist Finalisasi

- Ganti nama, NPM, program studi, pembimbing, kota, tanggal, dan panitia pada preamble.
- Tambahkan logo kampus bila diwajibkan.
- Isi versi library, hardware, model Ollama, dataset, dan protokol eksperimen yang benar-benar digunakan.
- Isi hasil numerik dan screenshot implementasi sebelum sidang.
- Periksa semua sumber dan sesuaikan format dengan pedoman kampus.
- Kompilasi dua atau tiga kali di LyX agar daftar isi, daftar tabel, daftar gambar, dan bibliografi diperbarui.